Monte Carlo Simulation of a Simple Polymer Chain

Implementation Report

Project_2026_II

12 April 2026

Contents

0.1 Project Overview

In this assignment, our goal was to develop a Monte Carlo simulation program to explore the conformational landscape of a 500-Carbon linear polymer chain (polyethylene) with varying torsional angles. The simulation generates the initial conformation (with & without Hydrogens), measures torsional energy rotations, end-to-end distance, and intramolecular Lennard-Jones interactions, and finally analyzes & visualizes the results.

After creating a serial processing version of the program, parallelization techniques involving both MPI and OpenMP were incorporated. These were compared to the serial version to demonstrate both their individual and combined High Performance Computing capabilities & benefits.

Many techniques taught in class were crucial in the development of this program, including: - **Git/Github** as a code respository & teamwork coordination platform - **Makefile** to automate the compilation and execution of the code - **Shell scripting** for facilitating server deployment - **MPI & openMP** to parallelize the code - **Python** for post-processing and plotting

0.2 Repository Structure

Github was used as our team's code repository management system, and from the root of the project, the code is divided among 3 main folders: `src`, `results`, and `docs`.

- `src` contains:
  - Makefile & shell scripts
  - Various versions of the main program (serial & parallel)
  - A `confs` folder where initial configurations are stored and compile-time options can be controlled via a file called `input.dat`
  - A `lib` folder where module source files, python analysis files, and plotting style dependencies reside (*further detail below*)
- `results` contains the output data from the simulations
- `docs` contains documentation and reports, including `img` folder for embedded images

In addition to these folders, a customary `README` file is included in the root directory detailing run instructions and code dependencies/requirements. The bulk of the code is written in Fortran, while the post-processing is done in Python.

Module Library Introduction. The following modules support the *`main...f90`* files:

- `energy.f90` & `energy_all_atoms.f90` — Permit **dual energy modes**, computing energetic contributions using either TraPPE-UA or OPLS-AA (with effective backbone potentials) depending on the `explicit_h` toggle dynamically read from `input.dat`.
- `initial-conf.f90` — Generates initial configuration of the polymer chain.
- `io.f90` — File input/ouput module.
- `monte_carlo.f90` — Provides the rotation algorithm and subroutine MCS step used to update spatial configurations.
- `observables.f90` — Computes the structural observables: End-to-end distance (R_{ee}), Radius of Gyration (R_g), and torsion angles.
- `parameters.f90` — Stores global parameters abstracted from *`main...f90`* code. Works in tandem with `input.dat` settings.

- *plot_...py* files — Various scripts used for plotting observables and serial vs parallel code comparisons.
- `requirements.txt` — List of python module dependencies.
- `science.mplstyle` — stylized document used in plotting to control visualization theme.

0.3 Code Management via GitHub

The project leader reviewed pull requests, checked that incoming changes followed the project guidelines, and merged them into the main branch. Team coordination was handled through a shared WhatsApp group, where members reported progress and informed the rest of the group whenever new code had been merged.

In most cases, integration was straightforward. However, overlapping uploads from different team members occasionally produced merge conflicts. This led the team to adopt a consistent workflow based on stashing and rebasing before recommitting local work. The process used was the following:

1. Stash local uncommitted changes:

   ```
   git stash
   ```

2. Pull the latest version of the main branch:

   ```
   git checkout master
   git pull main master
   ```

3. Rebase the local feature branch onto the updated main branch:

   ```
   git checkout <fork-feature>/<fork-branch>
   git rebase master
   ```

4. Resolve conflicts and re-apply the stashed changes:

   ```
   git stash pop
   ```

5. Commit and push the rebased branch:

   ```
   git push <fork-remote> <fork-branch> --force
   ```

This workflow helped keep the repository history cleaner and reduced repeated integration issues during collaborative development.

0.4 Makefile

As the project evolved, the Makefile also became more structured. It was used to automate compilation, execution, and post-processing for both the serial and parallel versions of the code.

The main compilation variables are:

```
FC        = gfortran
FFLAGS    = -O2 -Wall -Wextra
OBJ_DIR   = ../bin
EXE       = $(OBJ_DIR)/main_serial.x
LIB_OBJ   = $(OBJ_DIR)/parameters.o \
            $(OBJ_DIR)/io.o \
            ...
MAIN_OBJ  = $(OBJ_DIR)/main_serial.o
```

```
NP          ?= 4
OMP_THREADS ?= 2
```

Their roles are summarized below:

- `FC`: the Fortran compiler, initially set to `gfortran`.
- `FFLAGS`: compiler flags; `-O2` enables optimization, while `-Wall -Wextra` activates additional warnings.
- `OBJ_DIR` and `EXE`: define the directory for compiled objects and the name of the executable.
- `LIB_OBJ` and `MAIN_OBJ`: list the object files needed for the library modules and the main program.
- `NP` and `OMP_THREADS`: define the number of MPI processes and OpenMP threads used in parallel executions. These values can be overridden from the command line, for example with `make run_parallel NP=7 OMP_THREADS=3`.

To support both serial and parallel builds, the Makefile checks whether the requested target contains the word `parallel`. If so, it verifies that the MPI compiler wrapper is available and then switches the compiler from `gfortran` to `mpif90`.

```
ifneq (, $(findstring parallel,$(MAKECMDGOALS)))
  MPI_BIN := $(shell command -v mpif90 2>/dev/null)
  ifeq ($(strip $(MPI_BIN)),)
    $(error "MPI Compiler is required but 'mpif90' was not found!")
  endif
  FC = mpif90
  ...
endif
```

Instead of writing an explicit build rule for every `.f90` file, a pattern rule is used:

```
$(OBJ_DIR)/%.o: lib/%.f90
    @mkdir -p $(OBJ_DIR)
    $(FC) $(FFLAGS) -J$(OBJ_DIR) -I$(OBJ_DIR) -c $< -o $@
```

This rule states that any object file in `bin/` can be built from its corresponding source file in `lib/`. The `-c` option compiles without linking, while `-J` and `-I` direct the compiler to store and search module files in the object directory.

The final executable is linked through the default target:

```
all: $(EXE)

$(EXE): $(LIB_OBJ) $(MAIN_OBJ)
    @mkdir -p $(OBJ_DIR)
    $(FC) $(FFLAGS) -o $@ $(LIB_OBJ) $(MAIN_OBJ)
```

Finally, convenience targets are declared as `.PHONY` rules:

```
.PHONY: all clean run figures pipeline

run: all
    @mkdir -p ../results
    $(EXE)

figures:
```

```
    python lib/plot_results.py

clean:
    rm -rf $(OBJ_DIR)/*.o $(OBJ_DIR)/*.mod $(EXE)

pipeline:
    $(MAKE) run
    $(MAKE) figures
```

These shortcuts make it possible to compile, execute, clean, and post-process the project with short commands such as `make run`, `make clean`, and `make pipeline`.

0.5 Shell Scripting

Shell scripting was used to make the runs compatible with the CERQT2 cluster environment and to automate benchmarking. The scripts initialize the runtime environment, load the required modules, and set variables such as `OMP_NUM_THREADS`, `NSLOTS`, and `MPLBACKEND=Agg` for stable non-interactive execution. They were also used to automate parameter sweeps, build explicit trajectory manifests for the observables analysis, and collect timing data into summary files for the MPI and OpenMP benchmarks.

A qsub submission script, `run_collab.qsub`, was also written during program testing to launch an individual portion of the simulation on the HPC cluster. This script loads the modules required by the target queue, selects a custom Makefile separate from the main program Makefile, passes the appropriate command-line compilation arguments, and finally launches the simulation with `mpirun`.

1 Initial Configuration Generation

1.1 Internal-Coordinate Builder

The `initial_conf` module is responsible for generating the starting conformation of the polyethylene chain. The builder uses a recursive internal-coordinate approach: each successive carbon atom is placed by specifying a fixed C–C bond length ($d = 1.54\,\text{Å}$), a fixed C–C–C bond angle ($\theta = 114°$), and a selectable dihedral angle ϕ. A hard-sphere overlap check with threshold $r_{\min} = 2.0\,\text{Å}$ is performed after each placement to avoid unphysical self-intersections during chain construction.

```
! Place carbon i based on the last two accepted positions
do i = 4, n_carbons
  ! Bond vector from i-2 to i-1 (normalised)
  b1 = coords(i-1,:) - coords(i-2,:)
  b1 = b1 / vnorm(b1)

  ! Normal to the C(i-2)-C(i-1)-C(i) plane
  b0 = coords(i-2,:) - coords(i-3,:)
  n_vec = cross(b0, b1)
  if (vnorm(n_vec) > 1.0d-8) n_vec = n_vec / vnorm(n_vec)

  ! Proposed new position using dihedral phi
  d_vec = bond_length * (                              &
      -cos_theta * b1                                  &
      + sin_theta * (cos_phi * n_vec                   &
                  + sin_phi * cross(b1, n_vec)))

  candidate = coords(i-1,:) + d_vec
```

```
  ! Overlap check against all previously placed atoms
  overlap = .false.
  do j = 1, i-1
    if (vnorm(candidate - coords(j,:)) < r_min) then
      overlap = .true.
      exit
    end if
  end do
  if (.not. overlap) coords(i,:) = candidate
end do
```

Listing 1: Core internal-coordinate placement loop in `initial_conf.f90`.

When explicit hydrogens are requested (`explicit_h = .true.`), two hydrogen atoms are appended for each internal CH_2 group and one for each terminal CH_3 group, giving $N_{\text{atoms}} = 3N_C + 2$ total atoms for a chain of N_C carbons. The hydrogen placement is based on a local tetrahedral frame: instead of placing the H atoms coplanar with the C–C–C backbone (chemically inconsistent), they are positioned out of the backbone plane at the correct tetrahedral angle of $\approx 109.5°$.

```
! For each internal carbon i (not first or last):
! Build local orthonormal frame {b_fwd, perp, out_of_plane}
b_fwd = coords(i+1,:) - coords(i-1,:)
b_fwd = b_fwd / vnorm(b_fwd)

! Perpendicular to backbone, in the C(i-1)-C(i)-C(i+1) plane
b_back = coords(i-1,:) - coords(i,:)
b_back = b_back / vnorm(b_back)
perp   = b_back - sum(b_back*b_fwd)*b_fwd
if (vnorm(perp) > 1.0d-8) perp = perp / vnorm(perp)

! Out-of-plane direction (cross product)
out_of_plane = cross(b_fwd, perp)
out_of_plane = out_of_plane / vnorm(out_of_plane)

! Place H1 and H2 symmetrically out of plane (tetrahedral geometry)
! cos(109.5 deg) ~ -1/3  =>  sin(109.5 deg) ~ sqrt(8/9)
h1 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp)  &
     + sqrt(8.0d0/9.0d0) * 0.5d0 * ( out_of_plane + b_fwd) )
h2 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp)  &
     + sqrt(8.0d0/9.0d0) * 0.5d0 * (-out_of_plane + b_fwd) )
```

Listing 2: Out-of-plane hydrogen placement for internal CH_2 groups.

1.2 Configuration Modes

Five distinct initial configuration modes have been implemented, each designed to probe a different region of the polymer's conformational space:

Configurations 1 and 5 span the near-trans region of phase space, whereas configurations 2 and 3 provide disordered starting points. Configuration 4 was added to test helical initial geometries. This variety is relevant when the simulation is tested at different temperatures: some initial states may lie closer to the equilibrated ensemble than others, reducing the effective burn-in time.

Table 1: Summary of implemented initial configuration types.

conf_type	Name	Description
1	All-trans (planar)	All dihedrals set to $\phi = 0°$; fully extended zigzag chain. Maximum end-to-end distance.
2	Random	Each dihedral drawn uniformly from $[-\pi, \pi]$; maximally disordered starting state.
3	RIS-like discrete	Dihedrals assigned with probabilities reflecting the Rotational Isomeric State model (trans, gauche$^+$, gauche$^-$).
4	Spring / helix	Progressive dihedral increment per bond, producing a helical backbone (proposed by ai-murphy).
5	Perturbed trans	Small random perturbation applied to the all-trans geometry; slight out-of-plane deviation while remaining close to the global minimum.

2 Input/Output Module

2.1 Parameter Parsing from `input.dat`

All simulation parameters are read from a plain-text file `input.dat` through the `io` module. The parser applies default values for any missing keyword, strips inline comments (lines beginning with `#`), and performs basic range validation. The following parameters are controlled from the input file:

Table 2: Simulation parameters read from `input.dat`.

Parameter	Default	Description
`n_carbons`	10	Number of backbone carbon atoms
`n_steps`	1,000,000	Number of MC production steps
`temperature`	300.0	Simulation temperature (K)
`conf_type`	1	Initial configuration mode (see Table 1)
`explicit_h`	.false.	Enable all-atom (explicit hydrogen) mode
`rng_seed`	42	Random number generator seed
`print_interval`	1,000	Steps between output writes
`max_delta`	0.5	Maximum dihedral rotation per MC step (rad)

Output is written in XYZ format: trajectories to `traj_`*`label`*`.xyz`, energies to `energy_`*`label`*`.dat`, and observables to `observables_`*`label`*`.dat`, where the label encodes the key simulation parameters (`n_carbons`, `conf_type`, `n_steps`, `temperature`) for unambiguous identification.

3 Energy Evaluation and Monte Carlo Engine

3.1 Energy Calculations

The energy evaluation modules, `energy.f90` and `energy_all_atoms.f90`, were designed and implemented to compute the total potential energy of the chain and the ΔE increments required by the Metropolis acceptance/rejection step.

3.1.1 United-Atom (UA) Model & TraPPE-UA

Trigonometric bypass in dihedral calculations.

The standard approach to evaluating a dihedral angle requires an `acos` or `atan2` call, both of which are transcendental functions with significant CPU cost relative to basic arithmetic. Instead, `compute_cos_dihedral` computes $\cos\phi$ directly from the dot product of the normal vectors of the two planes defined by the carbon backbone:

$$\cos\phi = \frac{\mathbf{n}_1 \cdot \mathbf{n}_2}{|\mathbf{n}_1|\,|\mathbf{n}_2|}, \qquad \mathbf{n}_1 = \mathbf{b}_1 \times \mathbf{b}_2, \quad \mathbf{n}_2 = \mathbf{b}_2 \times \mathbf{b}_3 \tag{1}$$

where $\mathbf{b}_i$ are successive bond vectors. A guard clause handles collinear atoms (degenerate normals) by setting $\cos\phi = 1$ as a fallback. The function is declared `pure`, signalling to the compiler that it has no side effects, which allows it to be called safely from within parallel regions and `forall` constructs.

```
! Snippet from energy.f90: Direct cosine calculation
pure function compute_cos_dihedral(r1, r2, r3, r4) result(cos_phi)
  ! ... variable declarations ...
  b1 = r2 - r1
  b2 = r3 - r2
  b3 = r4 - r3

  n1 = cross_product(b1, b2)
  n2 = cross_product(b2, b3)

  nn1 = sum(n1**2)
  nn2 = sum(n2**2)

  ! Guard against collinear atoms (zero-length normals)
  if (nn1 < 1.0d-28 .or. nn2 < 1.0d-28) then
    cos_phi = 1.0d0   ! default to trans
    return
  end if

  ! cos(phi) = (n1 . n2) / (|n1| * |n2|)
  cos_phi = dot_product(n1, n2) / (sqrt(nn1) * sqrt(nn2))
end function compute_cos_dihedral
```

Polynomial expansion for TraPPE-UA.

The TraPPE-UA torsional potential [2] is:

$$U(\phi) = c_1(1 + \cos\phi) + c_2(1 - \cos 2\phi) + c_3(1 + \cos 3\phi) \tag{2}$$

Substituting $y = \cos\phi$ and applying the identities $\cos 2\phi = 2y^2 - 1$ and $\cos 3\phi = 4y^3 - 3y$ gives a polynomial in y:

$$U(y) = c_1(1 + y) + c_2(2 - 2y^2) + c_3(1 - 3y + 4y^3) \tag{3}$$

Since $y = \cos\phi$ is already available from the cross-product step, the torsional energy is evaluated with only multiplications and additions – no further transcendental calls are needed in the MC loop.

3.1.2 Explicit Hydrogens and the OPLS-AA Force Field

The all-atom model requires handling a substantially more complex topology. The OPLS-AA force field [3] was adopted, with atom-specific Lennard-Jones parameters for carbon and hydrogen

($\sigma_{CC} = 3.50\,\text{Å}$, $\varepsilon_{CC} = 0.066\,\text{kcal/mol}$; $\sigma_{HH} = 2.50\,\text{Å}$, $\varepsilon_{HH} = 0.030\,\text{kcal/mol}$; cross-terms from geometric mixing rules).

Effective backbone torsional potential.

In OPLS-AA, rotating a single C–C bond changes nine distinct dihedral angles: one C–C–C–C, four C–C–C–H, and four H–C–C–H. Computing all nine at every MC step is expensive for a code designed to run millions of steps. An *effective backbone potential* was therefore constructed by summing the OPLS-AA Fourier coefficients for all nine dihedral types analytically, producing a single set of effective coefficients that reproduce the combined torsional energy from the C–C–C–C backbone coordinates alone:

$$c_1^{\text{eff}} = 0.870, \quad c_2^{\text{eff}} = -0.0785, \quad c_3^{\text{eff}} = 1.5075 \quad (\text{kcal/mol}) \tag{4}$$

The OPLS-AA Lennard-Jones parameters are taken from the framework of Sæther et al. [3]; the effective coefficient derivation is an original contribution of this work. The same polynomial form used for the UA model is then applied with these coefficients, so the all-atom torsional evaluation has the same arithmetic cost as the united-atom one.

The corresponding `torsion_single` function in `energy_all_atoms.f90` is:

```
! Snippet from energy_all_atoms.f90: OPLS-AA Effective Potential
    evaluation
pure function torsion_single(cos_phi) result(e)
  double precision, intent(in) :: cos_phi
  double precision :: e, y, y2, y3

  y  = cos_phi   ! corresponds to trans = -1, cis = 1
  y2 = y * y
  y3 = y2 * y

  ! OPLS-AA evaluation using trigonometric identities
  ! cos(2x) = 2*cos^2(x) - 1 => 1 - cos(2x) = 2 - 2y^2
  ! cos(3x) = 4*cos^3(x) - 3*cos(x) => 1 + cos(3x) = 1 - 3y + 4y^3
  e = opls_c1 * (1.0d0 + y)             &
    + opls_c2 * (2.0d0 - 2.0d0 * y2)    &
    + opls_c3 * (1.0d0 - 3.0d0 * y + 4.0d0 * y3)
end function torsion_single
```

Topology mapping and exclusion matrix.

Non-bonded Lennard-Jones interactions must be suppressed for atom pairs separated by fewer than four bonds (1-2, 1-3, and 1-4), since those interactions are already captured by the bonded energy terms. The initialisation routine `init_energy_topology` assigns each hydrogen to its parent carbon by geometric distance (C–H threshold 1.2 Å), records the backbone position index for every atom, and builds a global Boolean matrix `is_excluded`. Each pair is then looked up in $O(1)$ per call, avoiding any bond traversal during the MC loop.

The exclusion criterion uses a uniform backbone-position threshold: pair (i, j) is excluded when $|\text{backbone\_pos}(i) - \text{backbone\_pos}(j)| < 4$. For C–C pairs this is exact. For C–H and H–H pairs the actual bond count exceeds the backbone-position difference by one and two respectively, so the threshold over-excludes some 1-5 and 1-6 interactions. The torsional potential partially compensates, since its parameterisation implicitly encodes the conformational barriers arising from steric repulsion. The single-chain, vacuum context of the simulation further limits the practical impact relative to a condensed phase. Correcting the exclusion criterion to a bond-count-aware rule is left for future work.

Dynamic atom lists for ΔE.

At each pivot move only atoms downstream of the chosen bond change position. Recomputing the full $O(N^2)$ LJ sum is therefore unnecessary. Instead, the atoms are partitioned into `fixed_list` and `moved_list` by comparing old and new coordinate arrays:

```
! Snippet from energy_all_atoms.f90: Dynamic lists for delta E
n_fixed = 0
n_moved = 0
do i = 1, n_atoms
  if (sum((coords_new(i,:) - coords_old(i,:))**2) > 1.0d-12) then
    n_moved = n_moved + 1
    moved_list(n_moved) = i
  else
    n_fixed = n_fixed + 1
    fixed_list(n_fixed) = i
  end if
end do
```

Only cross-interactions between fixed and moved atoms are re-evaluated, reducing the ΔE cost to $O(N_{\text{fixed}} \times N_{\text{moved}})$. A secondary `pair_type_matrix` routes each valid pair to the pre-computed C–C, C–H, or H–H LJ parameters in a single array lookup.

3.2 Pivot Move and Monte Carlo Step

The conformational sampling mechanism used throughout the project is the pivot move. Rather than perturbing the whole chain diffusely, the algorithm selects one internal C–C bond and rotates the entire tail of the polymer as a rigid body around that bond axis. This generates large collective rearrangements and allows the chain to explore configuration space much more efficiently than local coordinate displacements.

The rigid-body rotation is implemented with Rodrigues' rotation formula. If $\mathbf{v}$ is the position of an atom relative to the pivot point, $\mathbf{k}$ is the unit vector along the selected bond, and θ is the random rotation angle, the rotated vector is

$$\mathbf{v}_{\text{rot}} = \mathbf{v}\cos\theta + (\mathbf{k} \times \mathbf{v})\sin\theta + \mathbf{k}(\mathbf{k} \cdot \mathbf{v})(1 - \cos\theta).$$

This construction preserves bond lengths and bond angles while applying a clean rigid-body rotation to the selected downstream segment.

3.2.1 Handling sp^3 Hybridisation in All-Atom Mode

In all-atom mode, the same geometric transformation must be applied not only to the carbon backbone but also to the hydrogens attached to every rotating carbon. Because these hydrogens represent an approximately tetrahedral sp^3 environment, any inconsistency between the carbon motion and the attached hydrogen motion would immediately distort the local geometry and generate unphysical configurations. The AA rotation routine therefore identifies the hydrogens attached to each moving carbon and applies the exact same pivot point, axis, and angle to them, using a C–H bond threshold of 1.2 Å:

```
if (explicit_h) then
  ! Hydrogens are stored after the carbon backbone in the coords array
  do i = n_carbons + 1, n_atoms
    do j = k + 1, n_carbons
      ! Identify if hydrogen i is bonded to the moving carbon j
      v = coords(i, :) - coords(j, :)
      if (vnorm(v) < 1.2d0) then
```

```
        ! Apply Rodrigues' rotation relative to the pivot point
        v = coords(i, :) - pivot
        dot_uv = sum(axis * v)
        v_rot = v * cos_p + cross(axis, v) * sin_p &
              + axis * dot_uv * (1.0d0 - cos_p)
        coords_new(i, :) = pivot + v_rot
        exit  ! Hydrogen found, move to the next atom
      end if
    end do
  end do
end if
```

3.2.2 The MC Step and Metropolis Criterion

Each call to `mc_step` performs one trial move. First, a random internal bond index k is drawn uniformly from $[1, N_C - 2]$, and a random rotation angle ϕ is sampled uniformly from $[-\Delta\phi_{\max}, \Delta\phi_{\max}]$; the parameter `max_delta` is tuned to achieve a reasonable acceptance rate. Second, a trial configuration is generated by rotating the corresponding tail segment while leaving the current coordinates unchanged. Third, the change in energy is computed using the model-appropriate ΔE routine: `delta_energy_aa` for all-atom mode and `delta_energy_ua` for united-atom mode, dispatched via the `explicit_h` flag. Fourth, the trial move is accepted or rejected using the Metropolis criterion.

The Metropolis rule is standard. If the move lowers the total energy, it is accepted unconditionally. Otherwise, it is accepted with probability

$$P_{\text{acc}} = \exp(-\beta \Delta E), \qquad \beta = \frac{1}{k_B T}.$$

If accepted, the trial coordinates replace the current coordinates and the total, Lennard-Jones, and torsional energies are updated in place. If rejected, the trial geometry is discarded and the original state is retained. Repeating this procedure generates a Markov chain whose stationary distribution is the desired Boltzmann distribution.

```
! d. Metropolis acceptance criterion
call random_number(random_value)
if (dE < 0.0d0 .or. random_value < exp(-beta * dE)) then
  coords  = coords_new
  E_total = E_total + dE
  E_lj    = E_lj    + dE_lj
  E_tors  = E_tors  + dE_tors
  accepted = .true.
end if
```

3.3 Geweke Convergence Detection

Detecting equilibration in a Markov chain is non-trivial because successive configurations are strongly autocorrelated. Earlier versions of the workflow relied on a fixed burn-in length and a posterior post-processing check based on a Fast-Fourier Transform (FFT) script to estimate the integrated autocorrelation time τ_{int}, but this was often too rigid and could either discard too much data or fail to detect persistent transients, sometimes reporting that the production portion was 0% of the 10M steps. To make equilibration detection adaptive, the serial Monte Carlo engine was extended with an in-simulation Geweke diagnostic [1].

The Geweke test compares the mean of an early window of the trajectory with the mean of a late window of the same trajectory. In our implementation, the monitored quantity is the

total energy. If the chain has equilibrated, both windows should be consistent with the same stationary distribution. The resulting z-score is

$$z = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{\mathrm{SE}_A^2 + \mathrm{SE}_B^2}},$$

where $\bar{x}_A$ and $\bar{x}_B$ are the mean energies in the early and late windows, and SE_A and SE_B are their standard errors.

A crucial point is that the standard errors cannot be estimated from the raw sample variance as if the measurements were independent. Monte Carlo trajectories are correlated, so such a treatment would underestimate the uncertainty and produce false positives. To account for this, the code uses batch means. The window is partitioned into contiguous blocks, the mean of each block is computed, and the variance of those block means is used as an estimator of the uncertainty in the presence of autocorrelation:

$$SE_A^2 = \frac{1}{b_A(b_A - 1)} \sum_{i=1}^{b_A} (\bar{x}_i - \bar{x}_A)^2 \tag{5}$$

where $\bar{x}_i$ is the mean of block i and $\bar{x}_A$ is the mean of the full window. SE_B^2 is computed analogously over window B.

The implementation uses an accumulated buffer of $N = 300$ energy samples. The early window contains the first $f_A = 10\%$ of the buffer ($n_A = 30$ samples, $b_A = \lfloor\sqrt{30}\rfloor = 5$ blocks) and the late window contains the last $f_B = 50\%$ ($n_B = 150$ samples, $b_B = \lfloor\sqrt{150}\rfloor = 12$ blocks). Because block means of sufficiently large blocks behave approximately as independent, this estimator correctly absorbs the autocorrelation structure of the chain.

Under the null hypothesis of stationarity, z follows a standard normal distribution. Equilibrium is declared when $|z| < z_{\mathrm{crit}} = 1.96$ (95% confidence level) on $n_{\mathrm{consec}} = 3$ consecutive evaluations, preventing false positives caused by temporary plateaus. The test is re-evaluated every `eval_freq` synchronisation points once the buffer is full, so the simulation can stop the equilibration phase as soon as a statistically stable regime has been reached. The production phase then natively restarts step counting from 1.

3.4 Main Program Workflow Notes

The serial driver was updated so that equilibration is no longer treated as a hard-coded number of initial steps. Instead, the program reads the simulation parameters from `input.dat`, builds the initial coordinates, and enters a dedicated equilibration phase that continues until the Geweke criterion is satisfied. If a previously equilibrated geometry is already available, it can be loaded directly to bypass the initial search.

The main equilibration loop is a `do while (.not. is_equilibrated)` structure. At each step the code updates β, performs one MC trial move, and increments acceptance statistics when appropriate:

```
do while (.not. is_equilibrated)
  istep = istep + 1

  ! Annealing:
  if (istep <= n_steps) then
    T = T_ini - dT * dble(istep - 1)
  else
    T = T_fin
```

```
    end if

    ! Monte Carlo Step
    call mc_step(n_carbons, n_atoms, coords, symbols, explicit_h, &
                 beta, max_delta, E_total, E_lj, E_tors, accepted_step
                    )

    if (accepted_step) total_accepted = total_accepted + 1

    ! Output periodically
    if (mod(istep, print_interval) == 0 .or. istep == 1) then
      ! Energies
      write(u_ener, '(I10,␣3F15.4)') istep, E_total, E_lj, E_tors

      ! Observables
      rg2  = compute_rg(n_carbons, coords)
      ree2 = compute_end_to_end(n_carbons, coords)
      write(u_obs, '(I10,␣2F15.4)') istep, sqrt(rg2), sqrt(ree2)
      ...
```

- `do while (.not. is_equilibrated)`: Replaces traditional hard-coded burn-in periods with a dynamic loop that stays alive until equilibration is met.
- `T = T_ini - dT * dble(istep - 1)`: Retained hook for simulated annealing; the project simulations themselves run at fixed temperature with `T_ini` = `T_fin`.
- `call mc_step(...)`: Subroutine to perform a single Monte Carlo step.
- Observables (energy, radius of gyration, end-to-end distance, trajectory, etc.) are written only at `print_interval` checkpoints to avoid excessive I/O.

Once the Geweke criterion has been satisfied for the required number of consecutive checks, the equilibration phase ends. The equilibrated geometry is saved and used as the starting point for the production phase, whose step counter and output files are kept cleanly separate from the equilibration data. The parallel version of the program implements a much more sophisticated workflow, separating tasks between a main orchestrator and worker nodes; this is discussed in further detail in the Parallelization Techniques section.

4 Serial Driver, Results and Post-Processing

4.1 Two-Phase Simulation Structure

The updated serial workflow is built around a two-phase execution model: an adaptive equilibration stage followed by a fixed-length production stage. This is implemented in `main_serial_equil.f90`, where the program first reads `input.dat`, constructs the initial polymer geometry, and then decides whether equilibration should be performed from scratch or bypassed through a previously saved equilibrated configuration.

During Phase 1, the simulation performs Monte Carlo trial moves until the Geweke convergence diagnostic declares that the energy time series has reached a stationary regime. This replaces the old practice of assigning an arbitrary burn-in length in advance. Once equilibration is detected, the corresponding geometry is written to disk and becomes the starting point for Phase 2.

Phase 2 is the production run proper. Here, the serial driver resets the step counter and performs the requested number of Monte Carlo steps starting from the equilibrated geometry. In the serial reference case used throughout the report, this corresponds to a 10,000,000-step production simulation for a 500-carbon polyethylene chain with explicit hydrogens at 300 K.

The driver records its scientific output periodically through the `print_interval` mechanism. At each output checkpoint, the code writes the total, Lennard-Jones, and torsional energies together with the main structural observables, especially the radius of gyration R_g and the end-to-end distance R_{ee}. Torsion-angle data and trajectory snapshots are also written so that the structural evolution can be visualised afterwards.

An annealing schedule is still retained in the serial code in the form of a generic $T_{\mathrm{ini}} \rightarrow T_{\mathrm{fin}}$ update. For the simulations discussed here, however, the temperature is held fixed at 300 K, so this block acts mainly as a configurable extension point rather than a physically active part of the production protocol.

As a performance reference, the serial code was run on a single core of an Intel Core i7-11800H processor. In the benchmark comparison used in the report, one full serial production run required 8 hours, 18 minutes, and 31.91 seconds. This timing serves as the baseline against which the parallel implementations were later compared.

4.2 Automated Equilibration Detection

The serial results are post-processed through `plot_results.py`. This script begins by parsing `input.dat` so it can detect whether the run used explicit hydrogens and therefore decide which theoretical torsional potential should be overlaid in the final histogram plots. It then reads the separate equilibration and production outputs and stitches them together into a single coherent visual timeline.

For the energy and observable plots, the script concatenates the two phases and marks the transition with a vertical dashed line. This makes the time-demarcation between equilibration and production explicit and allows the reader to assess whether the dynamics truly become stationary after the detected equilibration point. By contrast, the torsional distribution is generated using only the production portion, since that is the only phase intended to represent equilibrium sampling.

The plotting layer is organized around three main functions: `plot_energies`, `plot_observables`, and `plot_torsions`. The first plots the total, Lennard-Jones, and torsional energies as functions of Monte Carlo step. The second plots R_g and R_{ee}. The third constructs the torsional histogram and superposes the corresponding theoretical torsional potential using the appropriate coefficient set for either the TraPPE-UA or OPLS-AA effective model.

Before the runtime Geweke detector was introduced, a post-processing equilibration analysis was also used as a diagnostic layer. In that approach, the Python workflow estimated the integrated autocorrelation time τ_{int} from the energy series by computing the autocorrelation function through an FFT-based convolution. This provides a fast way to estimate correlation lengths and effective sample sizes even for long trajectories.

That legacy Python analysis remains useful as a cross-check because it gives a purely post hoc view of correlation decay, burn-in length, and statistical inefficiency. However, it was not always robust enough to control the simulation by itself. In practice, long correlated plateaus could distort the variance estimates and lead to pathological truncations, including cases where the inferred production segment became unreasonably short. This limitation was one of the main reasons for moving equilibration detection into the Fortran runtime loop via the Geweke criterion.

The final post-processing pipeline therefore combines two roles. The Fortran code performs the actual online equilibration detection and separates the simulation cleanly into equilibration and production, and the Python code acts as the analysis and visualization layer that verifies, displays, and interprets the resulting time series and distributions.

4.3 Serial Version Simulation Results

The serial reference simulation consists of a Geweke-terminated equilibration run followed by a 10,000,000-step production run for a 500-carbon polyethylene chain with explicit hydrogens at 300 K. The chain starts from a highly extended configuration generated by `conf_type = 4`.

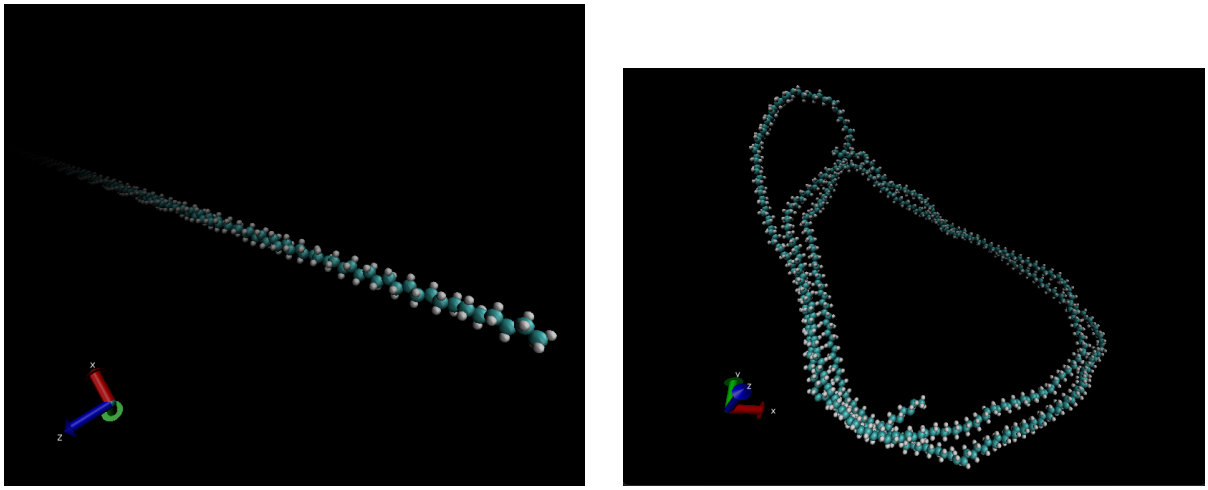

Figure 1: Left: initial configuration – a nearly fully extended chain with all backbone carbons approximately collinear. Right: final production configuration – a large open loop, characteristic of a random-coil state.

The two VMD snapshots in Figure 1 illustrate the scale of the conformational change. The initial chain is nearly straight, occupying roughly 650 Å end to end. The final structure is a large folded loop with the two chain termini close together, far more compact but not a dense collapsed globule.

4.3.1 Structural Observables

Figure 2 shows R_g and R_{ee} plotted continuously across equilibration and production. R_g begins at approximately 185 Å and collapses to ≈39 Å within the first 300 000 steps, after which it fluctuates by only ∼1 Å around its mean for the remainder of the run. R_{ee} follows a similar rapid collapse from ≈650 Å to ≈75 Å, but shows larger fluctuations in production (∼15 Å peak-to-peak), which is expected: unlike R_g, which averages over all atoms, R_{ee} depends only on the two terminal carbons and is therefore more sensitive to large-scale rearrangements of the chain ends.

Both observables had converged well before the Geweke criterion declared equilibrium at ≈3.9×10^6 steps.

4.3.2 Energy Evolution

Figure 3 shows the three energy components. At step 1, the total energy is approximately 200 kcal/mol, with the LJ term slightly negative around —25 kcal/mol (the extended chain has few close non-bonded contacts) and the torsional term near 230 kcal/mol. Within the first million steps the chain folds rapidly, driving the LJ energy from ≈−25 to ≈−150 kcal/mol as many atom pairs enter the attractive region of the LJ well simultaneously. The total energy settles to ≈50 kcal/mol (= −150 + 200 kcal/mol from the two components). The torsional energy changes comparatively little (230 to 200 kcal/mol), indicating that local torsional relaxation is much faster than global chain reorganization and that the initial configuration already had a broadly reasonable dihedral distribution.

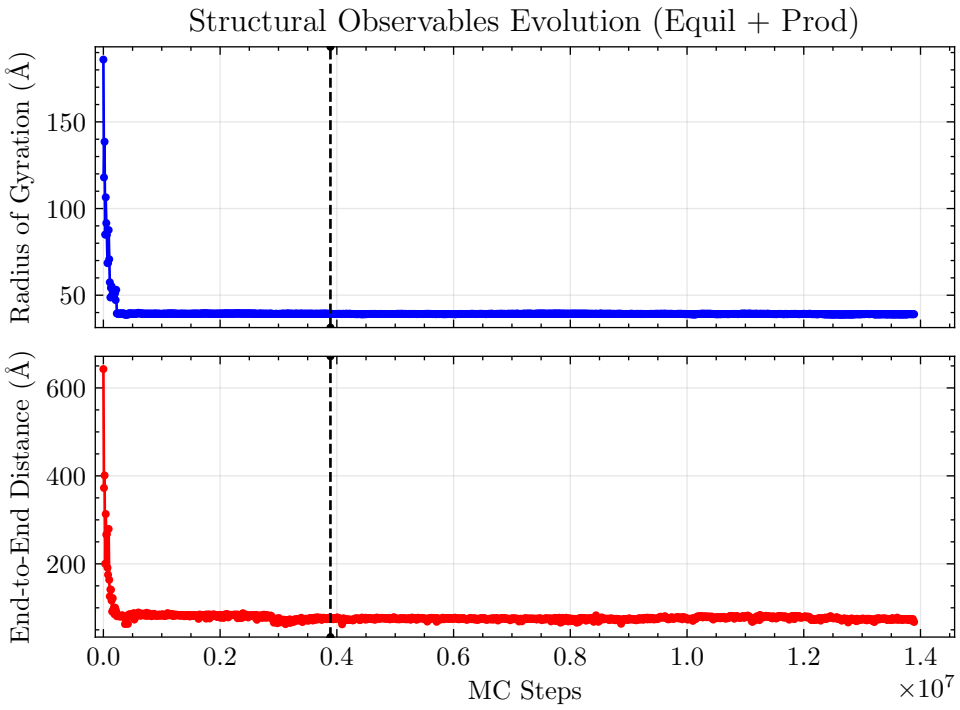

Figure 2: Radius of gyration R_g (top) and end-to-end distance R_{ee} (bottom) across equilibration and production. The dashed vertical line marks the Geweke equilibration boundary at $\approx 3.9 \times 10^6$ steps.

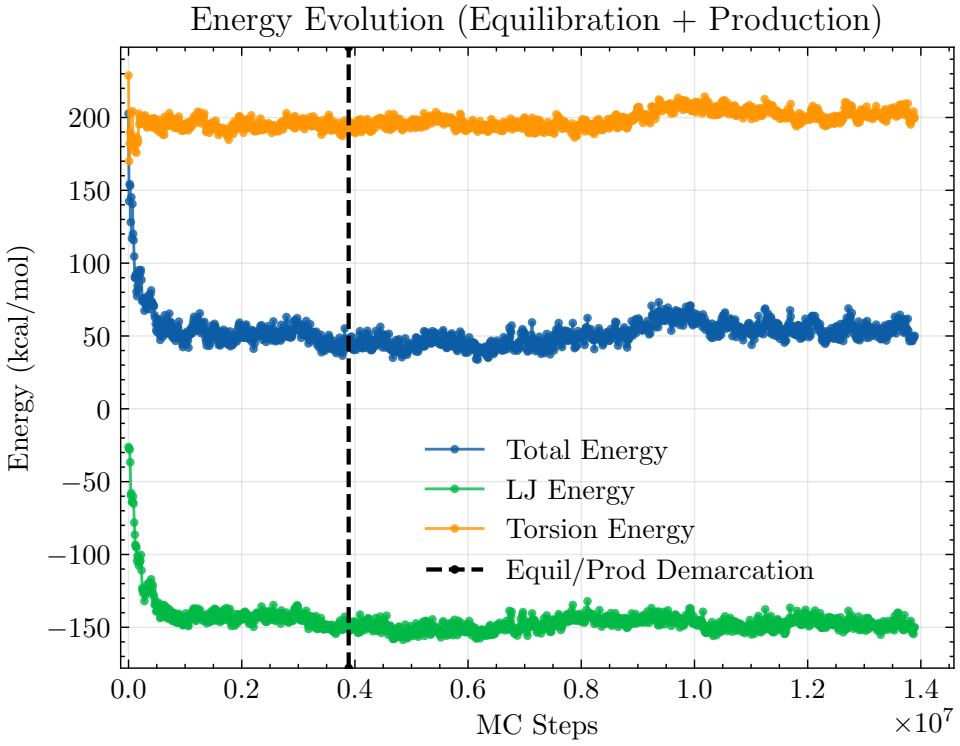

Figure 3: Total, Lennard-Jones, and torsional energy across equilibration and production. Dashed line marks the equilibration boundary.

After the equilibration boundary all three quantities fluctuate stably, with no systematic drift visible over the full 10 million production steps.

4.3.3 Torsion Angle Distribution

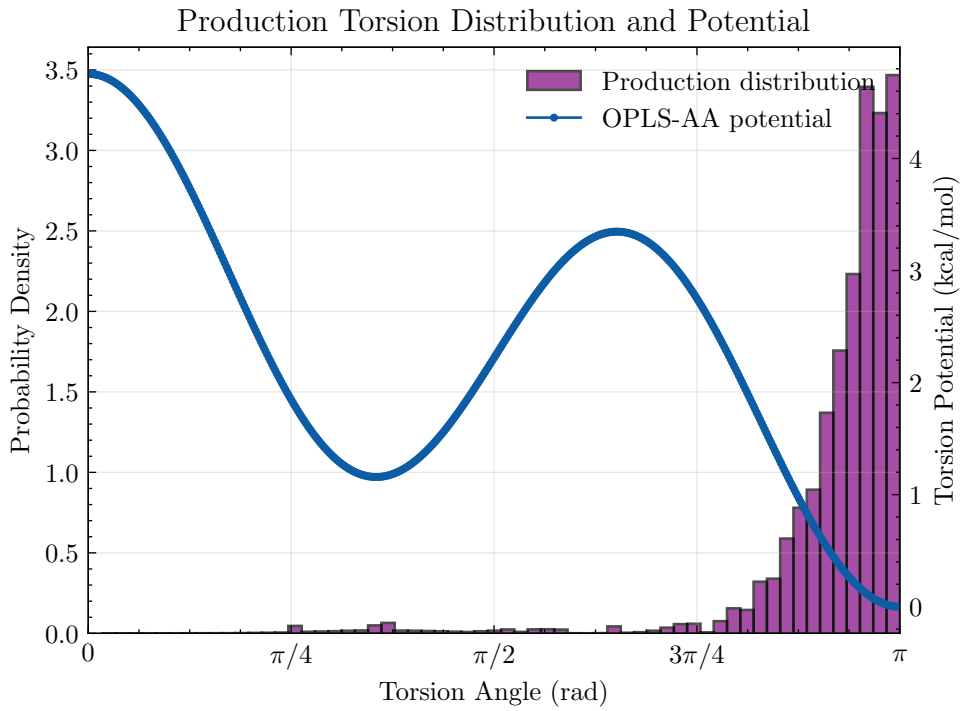

Figure 4: Production-phase torsion angle distribution (purple histogram) overlaid with the OPLS-AA effective backbone potential (blue curve, right axis).

Figure 4 shows the production torsion angle distribution alongside the OPLS-AA effective backbone potential. The histogram is concentrated almost entirely near $\phi = \pi$, with the rest of the angular range essentially unoccupied. A small gauche shoulder is visible near $\phi \approx \pi/3$, and the cis region ($\phi \approx 0$) is absent from the distribution. This population hierarchy is consistent with the potential: the global minimum is at $\phi = \pi$ ($U = 0$), the gauche well sits approximately 1.2 kcal/mol above it, and the cis barrier reaches approximately 4.7 kcal/mol – large relative to $k_B T = 0.59$ kcal/mol at 300 K.

The equilibrium torsional energy of approximately 200 kcal/mol visible in Figure 3 is consistent with this distribution. The torsional potential rises steeply from its minimum at $\phi = \pi$, so small thermal fluctuations within the trans well already carry a non-zero energy contribution per dihedral. Summed across all backbone dihedrals, these fluctuations account for the observed E_{tors} without requiring significant population outside the trans region.

4.4 Interpretation

The simulation results are consistent with expected polymer behaviour at 300 K. The chain collapses from its initial extended geometry within a few hundred thousand steps, after which it samples a stationary conformational ensemble – a compact random coil with $R_g \approx 39$ Å and $R_{ee} \approx 75$ Å. These values are substantially smaller than the fully extended length (≈650 Å) but much larger than a fully collapsed globule would give ($R_g \lesssim 15$ Å), confirming that the chain is in a random-coil rather than a globular state.

The dominant contribution to energy stabilisation upon folding is the Lennard-Jones term. In

the extended configuration, most atom pairs are beyond the LJ cutoff. As the chain folds, a large number of non-bonded contacts accumulate in the attractive r^{-6} region, releasing $\approx$125 kcal/mol of LJ energy. This is the thermodynamic driving force for collapse at 300 K.

The torsional distribution is governed by the total energy, not the torsional term in isolation:

$$E_{\text{tot}} = E_{\text{LJ}} + E_{\text{tors}}. \tag{6}$$

The Metropolis criterion accepts or rejects moves based on ΔE_{tot}. Despite this coupling, the resulting torsional distribution is dominated by the trans state in agreement with the torsional potential hierarchy, because the torsional barriers are large relative to $k_B T = 0.59$ kcal/mol at 300 K.

5 Cluster Compatibility

Portability to shared HPC infrastructure required two targeted modifications. First, the `Makefile` replaces the Fortran 2008 `open(newunit=u)` syntax with explicit unit-number declarations compatible with Fortran 90 compilers available on the CERQT2 cluster. Second, a submission script `1.run.sh` was created to load the appropriate GCC and OpenMPI modules before compilation and execution. The `parameters.f90` file was adapted to restore the `#ifdef` preprocessor block controlling the `explicit_h` compilation flag, and a comment was added to the `Makefile` explaining why the `-I$(MPI_INC_DIR)` flag is conditionally omitted in certain build configurations.

6 MPI Parallelization: Independent Replicas

6.1 Justification

The MPI parallelization implemented here is based on independent replica parallelism as a first step, while the energy evaluation and the Monte Carlo kernel can be parallelized subsequently (see the Energy Evaluation and Monte Carlo sections). First, independent replicas do not require inter-process communication inside the Monte Carlo loop, which keeps synchronization overhead very low. Second, running several initial configurations simultaneously makes it possible to assess how sensitive the final sampled ensemble is to the choice of starting geometry.

6.2 Implementation: `main_parallel_replicas.f90`

Starting from `main_serial.f90`, three MPI ranks are launched simultaneously. Each rank runs the identical Monte Carlo protocol (same thermodynamic parameters, same number of steps) but is assigned a distinct initial configuration and a perturbed random seed:

```
! MPI initialization
call MPI_Init(ierr)
call MPI_Comm_rank(MPI_COMM_WORLD, rank, ierr)
call MPI_Comm_size(MPI_COMM_WORLD, n_ranks, ierr)

! Assign initial configuration based on rank
select case (rank)
  case (0); conf_type = 1   ! all-trans (extended)
  case (1); conf_type = 4   ! helix / spring
  case (2); conf_type = 5   ! perturbed trans
end select

! Perturb the RNG seed so trajectories immediately diverge
rng_seed = rng_seed + rank * 104729
```

```
! Build the initial configuration and run the full MC protocol
call build_initial_conf(n_carbons, conf_type, explicit_h, coords,
    symbols)
call mc_run(...)

! Rank-labelled output so files do not overwrite each other
write(label, '(A,I1)') trim(base_label)//'_rank', rank
call write_trajectory(label, coords, symbols, n_atoms)
```

Listing 3: Rank-dependent configuration and seed assignment.

The three configurations chosen (`conf_type` 1, 4, and 5) cover the near-trans region from three distinct but close starting points: fully extended, helical, and slightly perturbed extended. This choice maximises the geometric diversity of the initial ensemble while keeping all replicas in a physically reasonable region of conformational space.

A serial counterpart, `main_serial_replicas.f90`, runs the same three configurations sequentially to provide a controlled performance reference. Both versions employ wall-clock timing: `MPI_Wtime` in the parallel driver and an equivalent system-clock call in the serial one, ensuring that the speedup ratios reported in Section 10 reflect actual elapsed time rather than CPU time.

6.3 Discarded Parallelization: XYZ Trajectory Writing

A second parallelization direction was explored: distributing the XYZ trajectory write step across ranks, so that each rank writes its own frame independently. The resulting wall-clock improvement was marginal (less than 2% of total runtime), confirming that trajectory I/O is not the computational bottleneck. The dominant cost lies in the energy evaluation and Metropolis acceptance/rejection logic, as expected from the $\mathcal{O}(N^2)$ Lennard-Jones loop. This branch was therefore discarded and the simpler rank-labelled sequential write was kept.

7 MPI Parallelization: Observable Post-Processing

7.1 Design and Implementation

Post-processing of the trajectory data has been parallelized in `main_parallel_observables.f90`. Rather than decomposing a single trajectory into contiguous frame blocks, the implementation operates at the file level: all trajectory files are first enumerated in a manifest, rank 0 reads this manifest, and the complete file list is broadcast to all MPI ranks. Each rank then processes a round-robin subset of trajectory files, independently computing the radius of gyration R_g, the end-to-end distance R_{ee}, and the torsional statistics for the configurations assigned to it.

This design was chosen because the available data are naturally distributed across multiple trajectory files, corresponding to different configuration types and simulation phases (`equil` and `prod`). A file-level distribution avoids repeated file seeks inside large XYZ trajectories, keeps the implementation simple, and preserves a fully embarrassingly parallel local workload. At the same time, it remains fully compatible with collective MPI communication for global aggregation.

A first collective step is performed through `MPI_Bcast`, which is used to distribute the manifest and the maximum number of torsional degrees of freedom to all ranks. After each rank has completed its local file subset, a mid-run checkpoint is evaluated by means of `MPI_Allreduce`. In this step, the partial frame counts and partial R_g sums are synchronised across all processes, so that the global partial mean can be computed before the final aggregation stage. This checkpoint was introduced as a lightweight example of active communication during the computation phase, beyond the final collection of results.

```
! Each rank processes a round-robin subset of trajectory files
do ifile = rank + 1, nfiles, num_procs
  call process_trajectory(trim(files(ifile)), max_phi, &
       local_file_count, local_frame_count, &
       local_sum_rg, local_sum_rg2, local_sum_ree, local_sum_ree2, &
       local_nphi, local_sum_phi, local_sum_phi2)
enddo

! Mid-run checkpoint: all ranks obtain the global partial mean of Rg
call MPI_Allreduce(local_frame_count, ckpt_frame_count, n_conf*n_phase
   , &
                   MPI_INTEGER, MPI_SUM, MPI_COMM_WORLD, ierr)
call MPI_Allreduce(local_sum_rg, ckpt_sum_rg, n_conf*n_phase, &
                   MPI_DOUBLE_PRECISION, MPI_SUM, MPI_COMM_WORLD, ierr
                      )

! Final aggregation on rank 0
call MPI_Reduce(local_sum_rg, global_sum_rg, n_conf*n_phase, &
                MPI_DOUBLE_PRECISION, MPI_SUM, 0, MPI_COMM_WORLD, ierr
                   )
```

Listing 4: Round-robin file distribution and MPI checkpoint in `main_parallel_observables.f90`.

The final global statistics are obtained on rank 0 through a series of `MPI_Reduce` operations, which combine the local sums, squared sums, frame counts, and torsional accumulators into global averages and standard deviations. The corresponding summary files (`summary_global_np1.dat`, `summary_global_np2.dat`, `summary_global_np4.dat`, and `summary_global_np8.dat`) show that the numerical results are identical for all tested values of n_p, confirming that the parallelization preserves the observable averages exactly.

The benchmark driver is invoked from the cluster submission script `3.run_parallel_observables_np.sh`, which executes the same post-processing workflow for $n_p \in \{1, 2, 4, 8\}$, records both MPI and shell wall-clock times, and writes per-process summary files for subsequent comparison.

8 MPI Parallelization: Collaborative Equilibration

8.1 Orchestrator Initial Configuration Read and Broadcast

In the MPI version of the program, rank 0 acts as the central orchestrator. Its role is to read the initial configuration files, decide whether each conformation must still be equilibrated or can directly enter production, and distribute the corresponding work to the workers.

At startup, the master reads the input settings and the geometry required to initialise each polymer configuration. If an equilibrated geometry is already available for a given configuration, the program can bypass the equilibration stage and directly unlock the corresponding production replicas. Otherwise, the master marks that configuration as pending collaborative equilibration.

For configurations that still require equilibration, rank 0 assigns a group of workers to the same initial structure. These workers all begin from the same geometry, but each one uses a different random seed so their trajectories immediately diverge. The master therefore provides a common starting point while still allowing independent exploration of phase space.

This orchestrator stage also centralises the campaign-level file handling. Workers do not need to parse the full simulation manifest or determine which configurations are ready for production.

Instead, they receive either the job descriptor or the coordinates required for the task they have been assigned, initialise their local arrays, and begin Monte Carlo propagation.

8.2 Equilibration Optimization

Rather than assigning a single worker to equilibrate each configuration type independently, a group of `workers_per_equil` MPI workers is assigned to each configuration type simultaneously. Each worker starts from the same initial geometry but with a different random seed, for example `rngseed = rank * 104729`, ensuring that trajectories immediately diverge and explore distinct regions of conformational space.

Every `sync_interval` Monte Carlo steps, a synchronisation protocol is executed. Each worker sends its current energy and R_g to the master, which selects the most representative configuration via a Boltzmann-weighted roulette wheel and broadcasts those coordinates back to the non-winning workers in the group. The winning worker continues its trajectory uninterrupted, preserving the best conformational path found so far.

The purpose of this strategy is to reduce the time required to reach a well-equilibrated low-energy configuration. Instead of waiting for one trajectory to escape a poor metastable region on its own, the algorithm lets several replicas search in parallel and periodically shares the best result across the group. In this way, all workers benefit from the progress made by the most successful replica.

8.3 Replica Selection: Boltzmann Roulette Wheel

At each synchronisation point, the master assigns a statistical weight to each replica i based on its total energy E_i:

$$w_i = \exp\left(-\frac{E_i - E_{\min}}{k_B T_{\text{virt}}}\right),$$

where $E_{\min}$ is the lowest energy in the current group and T_{virt} is a virtual temperature.

The virtual temperature is a purely algorithmic parameter with no physical meaning. It controls how aggressively low-energy replicas are favoured:

- $T_{\text{virt}} \to 0$: deterministic selection of the minimum-energy replica.
- $T_{\text{virt}} \to \infty$: uniform selection, ignoring energies.

A uniform random number then samples this distribution via roulette-wheel selection. The selected replica becomes the winner of that synchronisation round, and its coordinates are sent to the non-winning workers so they can resume the search from the most promising state found so far.

8.4 MPI Communication Protocol

The synchronisation uses only point-to-point MPI operations `MPI_Send` and `MPI_Recv` to avoid the need for temporary communicators or collective operations across worker subgroups. At each sync point, the master loops over all workers in the active equilibration group and performs the following exchanges.

Once the Geweke test declares equilibrium, the master saves the equilibrated geometry to `../results/equilibrated_cX.xyz` and unlocks the 10 production jobs for that configuration type in the task queue.

Step	Direction	Tag	Content
1	worker → master	`TAG_SYNC_OBS`	E_{tot}, R_g^2
2	master → worker	`TAG_SYNC_CTRL`	control signal, `bestlocalidx`
3	worker → master	`TAG_EQUIL_COORDS`	full coordinate array
4	master → non-winners	`TAG_SYNC_COORDS`	winning coordinates

Table 3: Point-to-point synchronisation protocol used during collaborative equilibration.

8.5 Metrics and Methodology

This section evaluates the performance of the equilibration optimization technique compared to the serial baseline. The system studied is configuration 1, a chain of 500 carbons at $T = 300$ K including hydrogens. Efficiency is evaluated based on the time to reach specific energy thresholds $E_{\text{tot}} = 110$, 100, and 90 kcal/mol.

The speedup is defined as

$$S_w = \frac{\text{Steps}_{\text{serial}}}{\text{Steps}_{\text{parallel},\, w}},$$

where w is the number of workers participating in the collaborative equilibration group. Efficiency is then defined as

$$\eta_w = \frac{S_w}{w}.$$

The table below summarises the number of MC steps required by different worker configurations to reach the target energy levels.

Threshold	Workers w	Steps Required	Speedup S_w	Efficiency S_w/w
110 kcal/mol	1	590,000	1.00	1.00
110 kcal/mol	2	70,000	8.43	4.21
110 kcal/mol	4	140,000	4.21	1.05
110 kcal/mol	8	50,000	11.80	1.47
100 kcal/mol	1	1,180,000	1.00	1.00
100 kcal/mol	2	350,000	3.37	1.68
100 kcal/mol	4	210,000	5.62	1.40
100 kcal/mol	8	50,000	23.60	2.95
90 kcal/mol	1	2,400,000	1.00	1.00
90 kcal/mol	2	1,780,000	1.35	0.67
90 kcal/mol	4	570,000	4.21	1.05
90 kcal/mol	8	160,000	15.00	1.87

Table 4: Monte Carlo steps required to reach selected energy thresholds during collaborative equilibration.

These results show that the method can exhibit superlinear speedup. This is not just because more workers are used, but because the search itself changes. A population of replicas explores several conformational regions at once, and once one replica finds a much better basin, that discovery is broadcast to the others at the next synchronisation point.

For this reason, the observed acceleration is partly computational and partly algorithmic. The collaborative protocol does not simply divide the same work among more processes; it improves the probability of finding favourable states earlier than a single serial trajectory would.

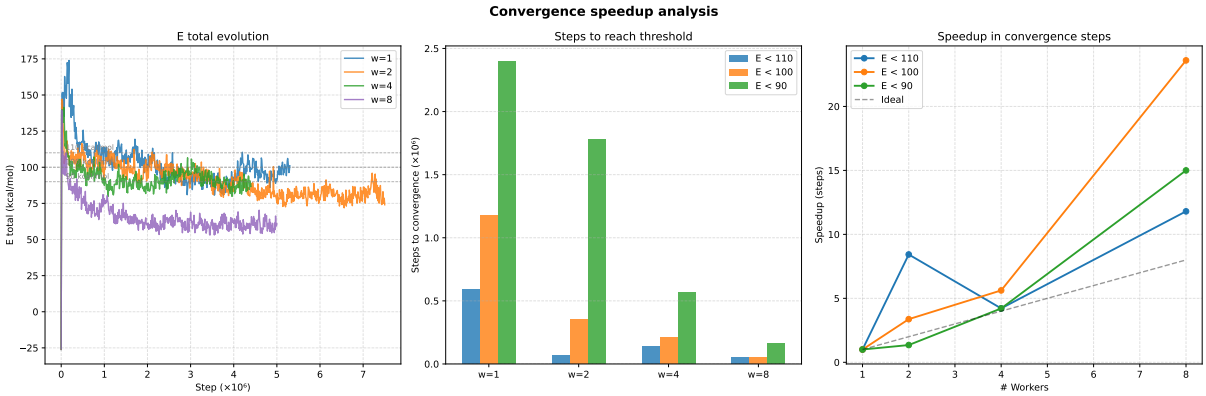

Figure 5: Convergence speedup analysis for collaborative equilibration.

8.6 Ensemble Averaging and Star Topology

Once an equilibrated geometry has been found, the workflow switches from collaborative equilibration to large-scale production sampling. At this stage, rank 0 acts as a central dispatcher in a star-topology MPI scheme, while the remaining ranks behave as workers that repeatedly request new tasks from the master.

The master maintains a global queue of production jobs. Each job corresponds to an independent production replica, and workers are assigned new jobs dynamically as soon as they finish their current one. This avoids the load imbalance that would appear if all replicas were distributed statically at launch, since individual trajectories may still differ slightly in runtime.

The dispatcher listens using a tag-based communication pattern and can react to whichever worker becomes free first. In practice, this means that workers request work, receive a production assignment, run the simulation, write their outputs, and notify the master when they have finished. The master then either sends another job or terminates the worker once the queue has been exhausted.

This star-topology design turns the MPI layer into an ensemble averaging engine. Independent production replicas are generated as efficiently as possible, and the final observables are obtained by averaging over the outputs of all completed runs. In this phase, the objective is no longer to accelerate the equilibration of a single chain, but to maximize statistical throughput across the whole ensemble.

Direction	Tag	Meaning
worker → master	`TAG_REQUEST_WORK`	worker requests a new task
master → worker	`TAG_DO_PROD`	start a production replica
master → worker	`TAG_WAIT`	remain idle temporarily
master → worker	`TAG_DIE`	terminate execution
worker → master	`TAG_PROD_DONE`	production replica finished

Table 5: Tag-based work distribution in the star-topology production phase.

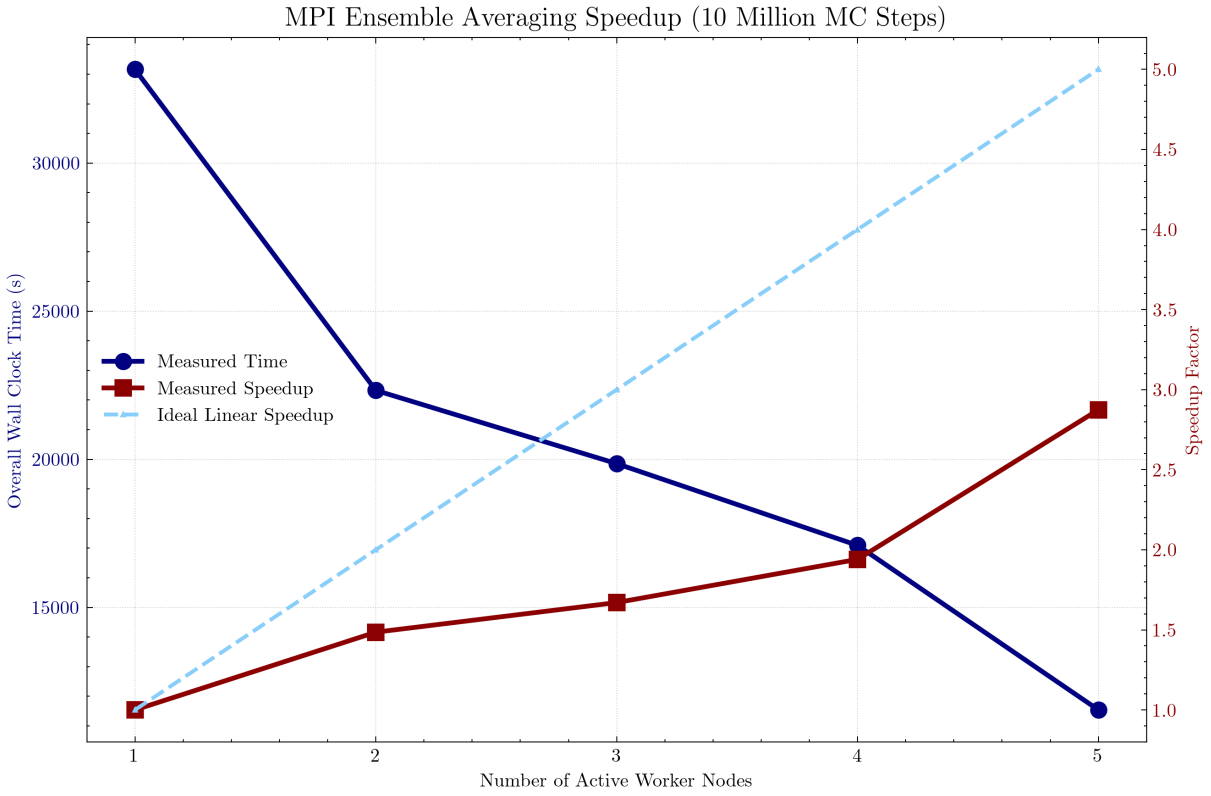

Figure 6: Timing and efficiency of the orchestrator-worker production workflow.

9 OpenMP Parallelization of Energy Routines

9.1 `compute_total_energy`: implementation

The $O(N^2)$ LJ pair loop in `compute_total_energy` has independent iterations and is suitable for OpenMP worksharing. Two separate `!$omp parallel do` regions were written: the LJ loop uses `schedule(dynamic, 10)` to compensate for the triangular iteration space (inner bound varies with `i`); the torsional backbone loop uses the default static schedule, which has lower overhead for a uniform workload. Private variables are declared per region (`j`, `r2` in the LJ region; `cos_phi` in the torsional region), and both accumulators (`e_lj`, `e_tors`) use a `reduction(+:...)` clause to prevent data races.

The parallelisation is activated at build time via the `_USE_OPENMP` preprocessor macro, which sets the `omp_total_energy` flag to `.true.`. This allows the same source to compile into a serial or parallel binary without code changes.

9.2 `delta_energy`: implementation

The fixed/moved partitioning reduces the LJ work to a rectangular $N_{\text{fixed}} \times N_{\text{moved}}$ loop. The `collapse(2)` directive merges both loop indices into a single flat iteration space, which OpenMP distributes evenly across threads. Because pivot moves near the chain termini produce very few moved atoms, a runtime guard `if(omp_delta_energy .and. (n_fixed*n_moved > 1000))` prevents thread-spawn overhead from exceeding the parallel benefit for small moves.

9.3 Benchmarking: `compute_total_energy`

A dedicated benchmark program performs 10 000 consecutive calls to `compute_total_energy` on a fixed geometry and reports the total wall time via `MPI_Wtime`. The script sweeps thread counts of 1, 2, 3, and 4 across chain sizes of $N = 20, 50, 100, 500$ carbons in all-atom mode. Figure 7 shows the wall times and parallel efficiency $E = T_1/(p \cdot T_p)$.

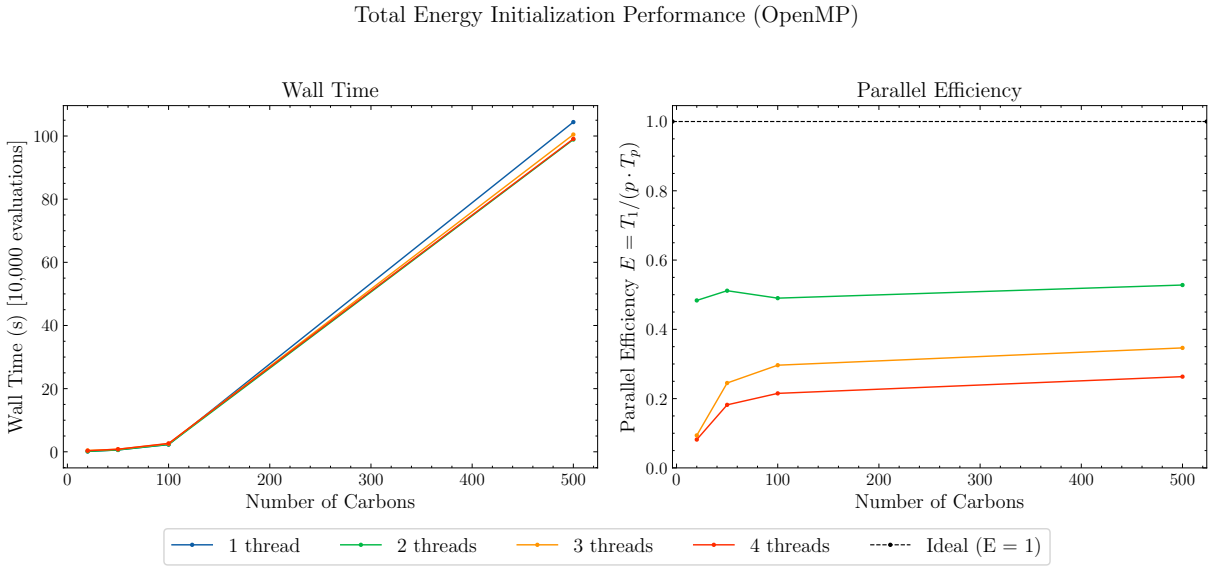

Figure 7: OpenMP scaling of `compute_total_energy` (10 000 evaluations, all-atom model, explicit hydrogens). Left: absolute wall time. Right: parallel efficiency $E = T_1/(p\,T_p)$; dashed line marks ideal efficiency $E = 1$.

Three features stand out. First, at small chain sizes ($N \leq 100$) adding threads increases wall time — at $N = 20$, three and four threads are slower than serial — confirming that the computation volume is too small to cover thread-management overhead. Second, even at $N = 500$ all thread counts converge to essentially the same wall time ($\approx$ 99–104 s), with efficiencies of 0.53, 0.35, and 0.26 for 2, 3, and 4 threads respectively – well below ideal, with only marginal improvement as N grows.. This indicates that the bottleneck is not arithmetic but memory bandwidth: the triangular LJ loop accesses coordinate data in a pattern that generates cache misses, so threads stall waiting for data rather than executing useful work. The `is_excluded` branch inside the inner loop further fragments the iteration space in a way that `schedule(dynamic, 10)` cannot fully correct.

`compute_total_energy` is called once at initialisation; the energy accumulators are subsequently updated incrementally via $E \leftarrow E + \Delta E$ at each accepted move, so no repeated full-energy evaluations occur during the MC loop. Given that parallelisation of this routine yields no measurable benefit, the `omp_total_energy` flag was disabled for all production simulations.

9.4 Benchmarking: `delta_energy`

Because `compute_total_energy` is called only at initialisation, the actual runtime cost of a 1 M-step simulation is dominated entirely by `delta_energy`. At $N = 500$ with one thread, the benchmark records 6,592 s for 10^6 steps, compared to 0.01 s for the single `compute_total_energy` call — a factor of $\sim$660,000 difference.

The `delta_energy` benchmark was run as a full 10^6-step MC simulation rather than an isolated call counter, since that is the context in which the routine operates in production. Three in-

dependent replicas were executed simultaneously as separate MPI processes, each starting from a different initial chain configuration. Wall time was recorded per replica using `MPI_Wtime()` and the maximum across all three ranks was taken as the reported time for each (thread count, chain size) combination. Thread counts of 1, 2, 3, and 4 were tested across chain sizes of $N = 50$, 100, 500 carbons. Figure 8 shows the results.

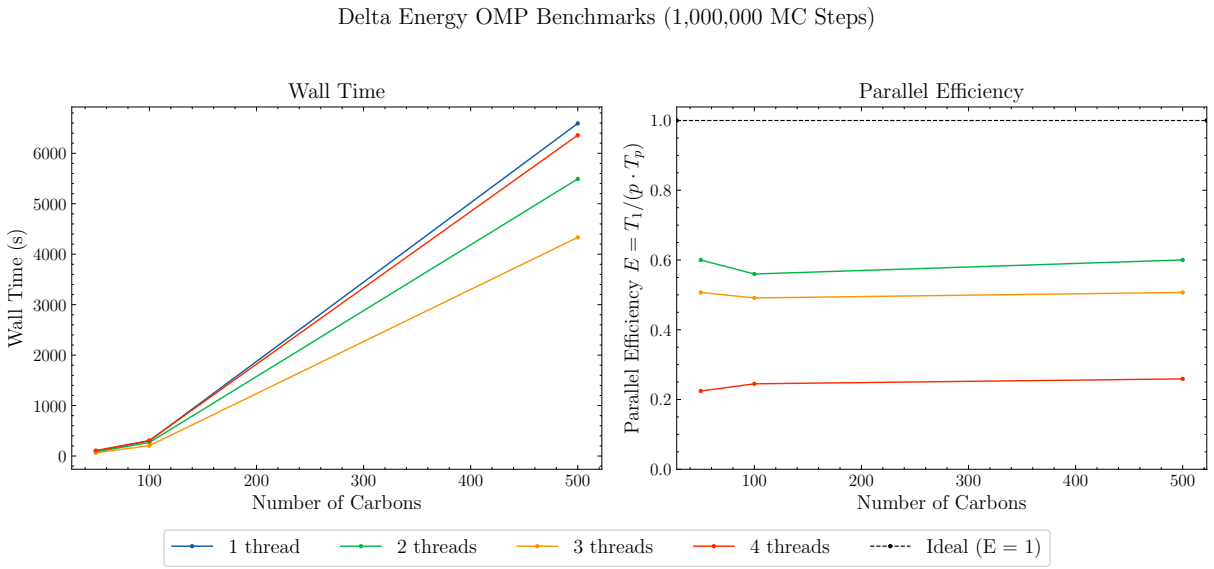

Figure 8: OpenMP scaling of `delta_energy` (10^6 MC steps, all-atom model). Left: absolute wall time. Right: parallel efficiency $E = T_1/(p\,T_p)$; dashed line marks ideal $E = 1$.

Unlike `compute_total_energy`, `delta_energy` shows genuine speedup with 2 and 3 threads. Efficiency with 2 threads is 0.60 at both $N = 50$ and $N = 500$, and 0.56 at $N = 100$, giving wall-time reductions of 11–17%. With 3 threads the speedup reaches 1.47–1.52× (efficiency ≈ 0.50) across all chain sizes, reducing the $N = 500$ simulation from 6,592 s to 4,333 s.

Four threads, however, is consistently counterproductive. At $N = 50$ and $N = 100$ the 4-thread run is slower than the serial baseline (speedup 0.90 and 0.98 respectively), and at $N = 500$ the gain is negligible (speedup 1.04×). The efficiency values of 0.22–0.26 across all sizes indicate that thread-management overhead outweighs the parallel benefit at this thread count.

Notably, the efficiency values are nearly independent of N. This is explained by the `is_excluded` branch inside the inner loop: the number of valid (non-excluded) pairs in the $N_{\text{fixed}} \times N_{\text{moved}}$ space depends on topology, not simply on N, so the effective parallel workload does not scale uniformly with chain length. The `collapse(2)` directive assigns the flat iteration range to threads before knowing which pairs will be skipped, leaving load imbalance that static scheduling cannot correct. For 4 threads, synchronization cost and this imbalance together erase the parallel benefit.

The practical conclusion is that 3 threads is the effective optimum for `delta_energy` (tested on the `cerqt01.q` cluster queue): it delivers a consistent ≈1.5× speedup with ≈50% efficiency, reducing the wall time for a 10^6-step simulation at $N = 500$ carbons from 6,592 s to 4,333 s — a 34% reduction. Four threads provides no benefit on this hardware, and is slower than serial for smaller chain sizes.

9.5 SIMD exploration

An attempt was made to exploit single-core SIMD vectorisation on `lj_pair_energy` with the `!$omp declare simd` directive, processing multiple atom pairs simultaneously in AVX registers.

The `cycle` on `is_excluded` inside the inner loop is incompatible with SIMD regions, and removing it to enable vectorisation degraded the load balance provided by `collapse(2)`. The directive was removed. Revisiting this is feasible if the pair loops are refactored to use pre-filtered pair lists, which would eliminate the exclusion branch entirely.

10 Scaling Analysis and Discussion

10.1 Replica Parallelism: Speedup vs. System Size and Run Length

The independent-replica MPI strategy was benchmarked by varying both the polymer size and the production run length. In this approach, each MPI worker executes a different production replica, so the total wall time depends primarily on how much useful work can be distributed across workers before communication and orchestration overhead begin to matter.

The first scaling study varies the number of carbons while keeping the simulation length fixed. As the chain becomes larger, each replica contains more internal work, especially in the energy calculations and observable evaluations, so the MPI orchestration overhead becomes less significant relative to the total runtime. For this reason, larger systems are expected to display better parallel efficiency than very small ones.

The second study varies the number of Monte Carlo steps for a fixed system size. Short runs do not amortize the startup, scheduling, and I/O costs very well, whereas longer runs allow each worker to spend a much greater fraction of time performing actual MC sampling. Consequently, speedup generally improves as the number of production steps increases.

In both cases, the speedup is not expected to be perfectly linear. The master node still performs administrative tasks such as initial file reading, dispatching jobs, receiving completion messages, and managing the production queue. This means that part of the total hardware budget is always reserved for orchestration rather than direct numerical work.

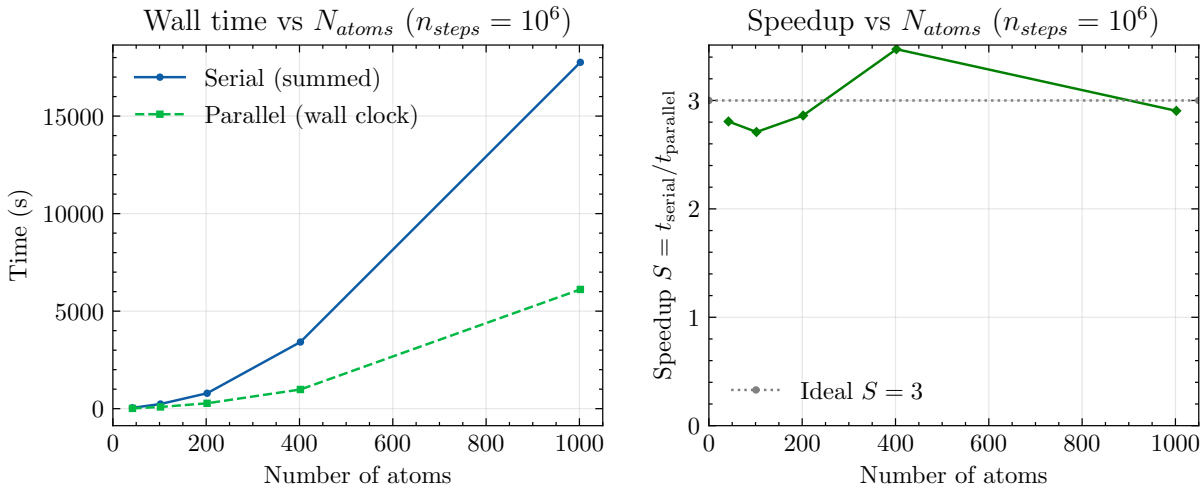

Figure 9: Speedup of independent-replica parallelization: speedup as a function of chain length ($N_C \in \{20, 50, 100, 200, 500\}$, $n_{\text{steps}} = 10^6$). All runs use 3 MPI ranks. The dashed line indicates ideal linear scaling ($S = 3$).

10.2 Observable Post-Processing: Strong Scaling

A second scaling study was carried out for the observable post-processing kernel, implemented in `main_parallel_observables`. Unlike the production simulations, this stage is a strong-scaling

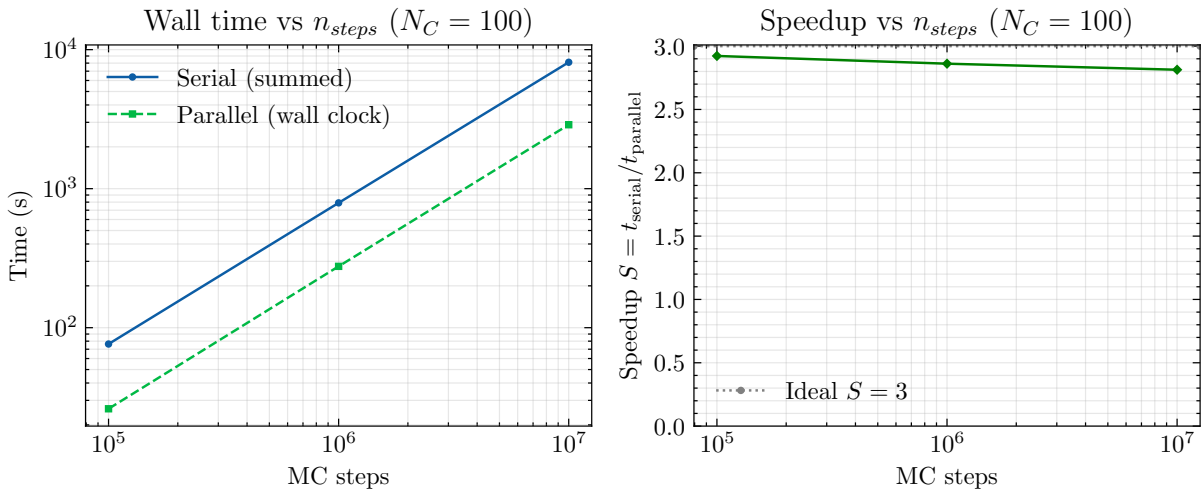

Figure 10: Speedup of independent-replica parallelization: speedup as a function of run length ($n_{\text{steps}} \in \{10^5, 10^6, 10^7\}$, $N_C = 100$). All runs use 3 MPI ranks. The dashed line indicates ideal linear scaling ($S = 3$).

problem: the total dataset is fixed, and the number of MPI processes is increased to measure how efficiently the same workload can be completed.

Each process reads a subset of trajectory files, computes the radius of gyration, end-to-end distance, and torsional statistics, and accumulates local partial sums. At the end of the run, the global results are assembled through MPI reductions on rank 0, which writes the final averaged observables.

This workload is well suited to MPI because each trajectory file can be processed independently for most of the runtime. Communication is only required at the end, when the local statistics are combined into a single global dataset. As a result, the observable analysis behaves much closer to an embarrassingly parallel workload than the simulation phase itself.

The scaling is therefore expected to be strong for moderate process counts, with efficiency gradually decreasing once the number of workers becomes large compared to the number of files. At that point, the amount of useful work per process shrinks and the fixed costs of process launch, file I/O, and final reductions become more visible.

10.3 Discussion

One of the key observations from our performance analysis is that parallelization rarely translates into perfectly linear speedup. In an ideal case, doubling the number of cores would halve the runtime, but in practice communication overhead, process coordination, file handling, and hardware contention all reduce the efficiency of additional workers.

This effect is visible in both the independent-replica and post-processing benchmarks. For the MPI production workflow, the master must coordinate task dispatch and receive status messages from workers. For the observables workflow, the amount of useful work per process eventually becomes too small compared to the fixed costs of launching processes and reducing the final results.

Resource utilization is therefore just as important as raw parallelism. The orchestrator-worker design helps maintain high CPU occupancy by ensuring that free workers are immediately assigned new tasks from the queue. At the same time, the staged execution model avoids wasting

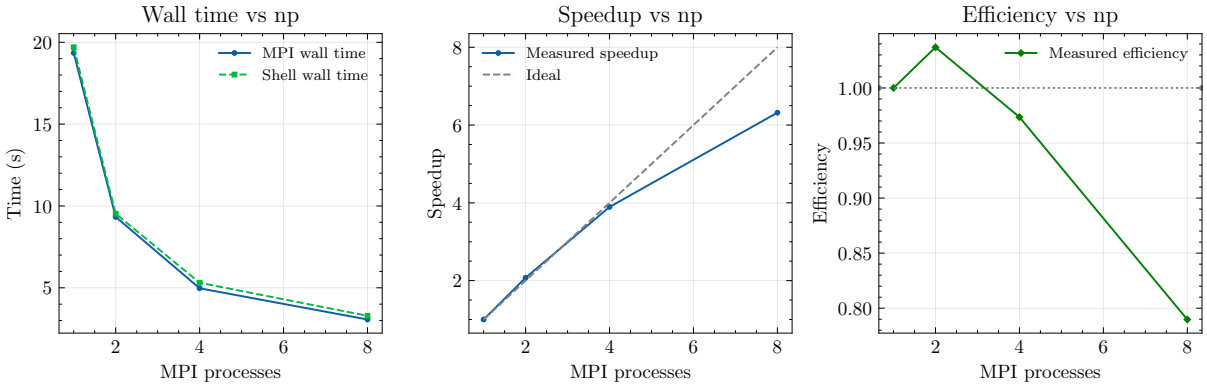

Figure 11: Strong scaling of the observable post-processing kernel. MPI wall-clock time (solid circles) and shell-measured elapsed time (dashed squares) as a function of the number of MPI processes n_p.

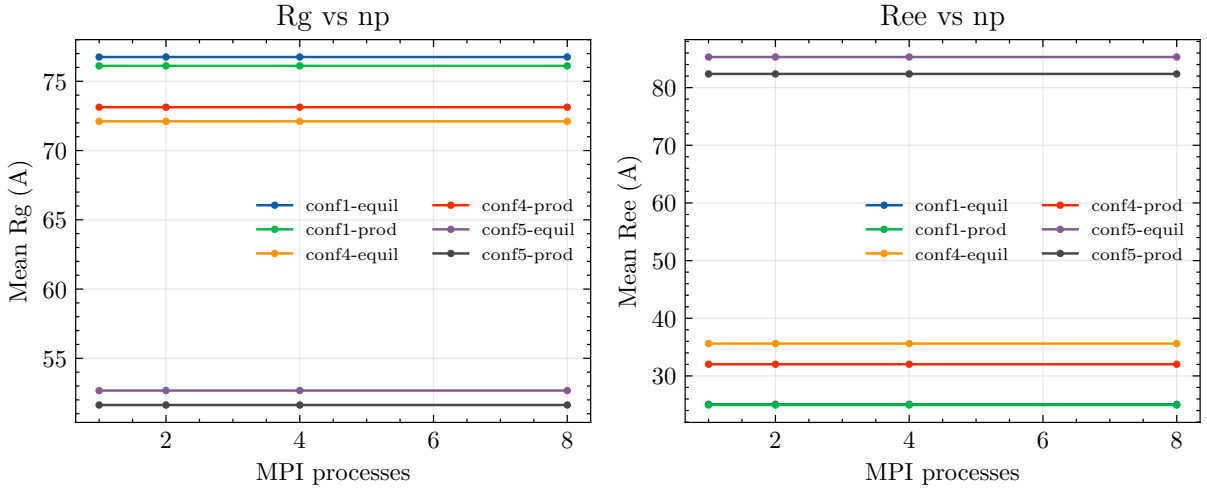

Figure 12: Mean radius of gyration R_g and mean end-to-end distance R_{ee} as a function of the number of MPI processes n_p. The overlap of all curves confirms that the observable averages are preserved exactly across the tested parallel decompositions.

resources on production jobs before equilibration has completed.

Combining MPI and OpenMP also requires careful control to avoid oversubscription. If too many MPI ranks each spawn too many OpenMP threads, the runtime can degrade due to thread collisions and operating system scheduling overhead. This is why the Makefile parameters `NP` and `OMPTHREADS` are treated as a coordinated resource-management mechanism rather than as independent tuning knobs.

Finally, the scaling trends also reflect the nonlinearity of Monte Carlo workloads themselves. Some replicas converge faster than others, some trajectories generate more favourable moved/-fixed partitions inside `delta_energy`, and some process counts match the available job granularity better than others. For this reason, the best-performing configuration is not always the one with the largest number of cores, but the one that best matches the size and structure of the workload.

10.4 Combined Parallel Workflow

The final program combines several levels of parallelism into a single coordinated workflow. At the top level, MPI is used to distribute work across processes in an orchestrator-worker star topology. Depending on the phase of execution, workers either participate in collaborative equilibration or execute independent production replicas drawn from a global queue.

Within each worker, OpenMP is used to accelerate the most expensive energy kernels, especially the nested Lennard-Jones loops in `compute_total_energy` and `delta_energy`. This creates a hybrid structure in which MPI handles macro-level task distribution while OpenMP performs micro-level acceleration inside each simulation task.

A key design choice is that the simulation is separated into phases so that resources can be reused dynamically. If an equilibrated geometry already exists, the master can bypass the equilibration stage for that configuration and immediately unlock the corresponding production jobs. Meanwhile, workers that finish equilibration for other conformations are reassigned to the shared production queue, ensuring that CPU resources are not left idle.

This combined architecture does not simply attempt to maximize the number of active cores at all times. Instead, it tries to maximize the amount of useful work completed per unit time while respecting the constraints of communication overhead, thread contention, and workload granularity. In this sense, the program acts as a coordinated HPC ecosystem rather than as a single parallel loop.

11 Parallel Simulation Results

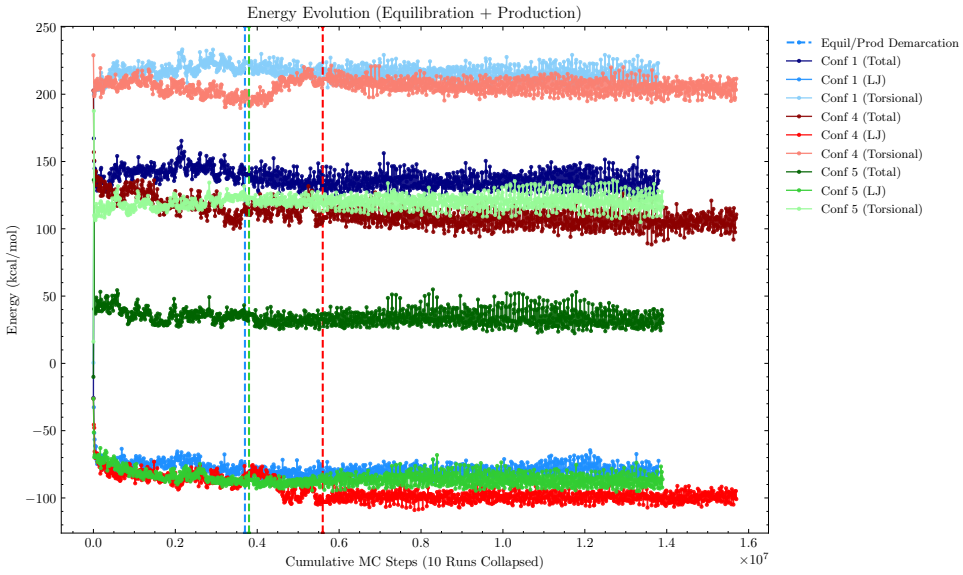

Figure 13: Total, LJ, & Torsional Energy plots across various initial conformations

Figure 13 tracks the total, Lennard-Jones (LJ), and torsional energy trends over the progression of Monte Carlo Steps (MCS) for configurations 1, 4, and 5. The vertical dashed lines mark the transition point between equilibration and the subsequent 10-million step production data.

In this particular instance of the combined simulation, the torsional and LJ energy took longer to stabilize in configuration 4 than the other configurations (1 & 5), meaning equilibration took longer to pass the Geweke equilibration determination test as indicated in the red, vertical, dashed line. For this reason its combined simultion to run for $\sim 1.55 \times 10^7$ total steps.

An important observation and detail obtained from this graph is how initial configuration influences the total energy within the simulation. Configurations 1 & 4 are rigid and straight helical structures, respectively, while introducing random, out-of-plane tilts to configuration 5 allowed it to find a much lower equilibrated torsional (and therefore, total) energy.

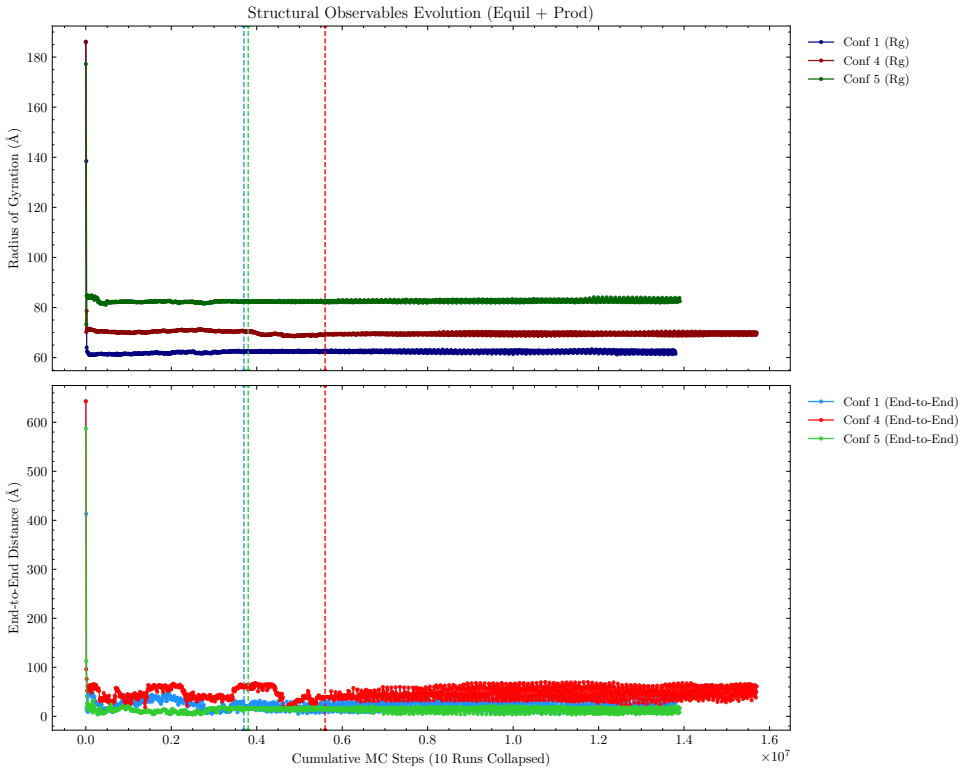

Figure 14: Radius of Gyration & End-to-End Distance plots across various initial conformations

Figure 14 displays the Radius of Gyration (Rg) and End-to-End Distance for the different configurations. As with the energy plots above which showed similar torsional energies between Conf 1 & 4 while Conf 5 was much lower after equilibration, the Rg for Conf 5 is the most distinct here as well. Configuration 5, due to its initial out-of-plane tilts, was able to better relax its bonds into low-energy rotational valleys, allowing the chain to naturally untangle and adopt an expanded, looser geometry, mathematically resulting in a higher Radius of Gyration.

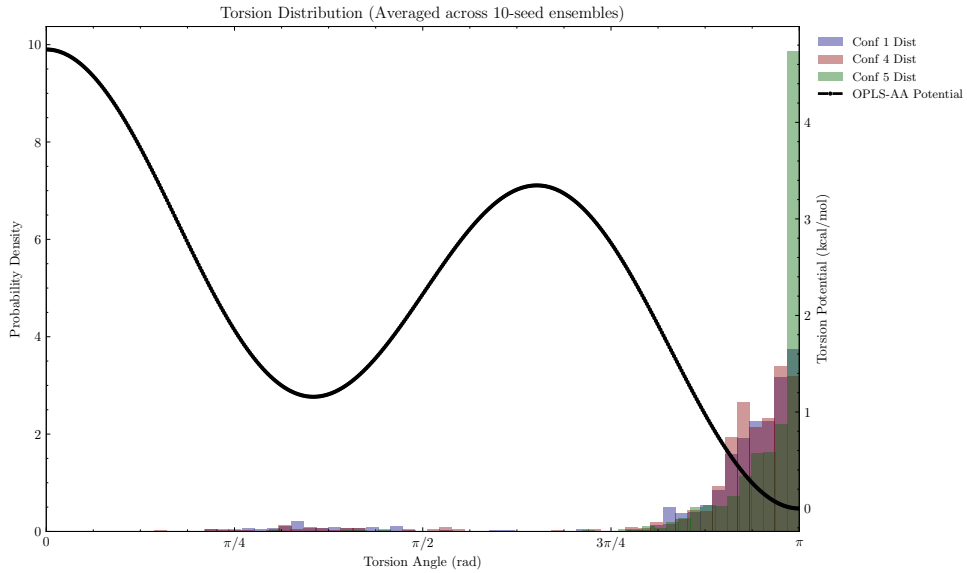

Figure 15: Torsional distribution of post-equilibrated MCS across various initial conformations

The low-energy rotational valley relaxation tendency for Conf 5 is even more apparent in figure 15, which shows the probability density of the each polymer exploring various dihedral (torsion) angles. Each configuration's post-equilibration data is compared to the OPLS-AA potential, where the results confirm the highest probability at π (trans) configuration. While all configurations follow the expected potential, Conf 5 adheres most closely, fully affirming the above energy and observable data. Also of note is how the graph demonstrates the expected rise in probablity around the potential's local minima at $\sim\pi/3$.

12 Conclusions

12.1 Computer Simulation vs. Real-World Results

While this Monte Carlo simulation successfully modelled the conformational sampling of a polymer chain, it is important to recognise the limitations when comparing theoretical calculations to real-world experimental results.

- By employing the OPLS-AA and TraPPE-UA force fields, we gain robust mathematical approximations of energy landscapes; however, these empirical potentials cannot fully capture the dynamic complexities of real-world solvent interactions or subtle quantum effects.
- The isolation of a single polymer chain within the simulation neglects the significant impact of intermolecular forces and environmental factors that would be present in a real-world scenario.
- The all-atom energy module applies a uniform backbone-position threshold to suppress short-range non-bonded interactions, which over-excludes some 1-5 and 1-6 hydrogen–hydrogen contacts. The impact is partially mitigated by the torsional potential and the single-chain, vacuum context of the simulation. Nonetheless, this represents a known approximation whose correction is left for future work.

12.2 Nonlinearity of Parallel Speedup

One of the key observations from our performance analysis is that parallelisation rarely translates evenly into a 1:1 linear speedup. In a perfect scenario, assigning four cores would make

the code run four times faster. However, as seen in our MPI benchmarks, adding more workers inherently introduces communication overhead. The time required for the orchestrator to pass messages, synchronise coordinates during ensemble sampling, and track dynamic convergence tests gradually eats into the raw processing speed. At certain hardware boundaries, communication bottlenecks and competition for physical system resources make additional core count benefits negligible.

12.3 Navigating Resource Utilization

Combining multiple High-Performance Computing (HPC) methodologies – such as MPI and OpenMP – offers significant acceleration capabilities, but requires careful resource management to prevent oversubscription. Breaking up the simulation workflow into phases proved to be a critical step in reusing resources and limiting the system size required to run the program.

12.4 HPC Optimization

Ultimately, achieving high performance is not simply a matter of checking whether a particular parallel processing technique outpaces its serial counterpart; rather, it requires a coordinated benchmarking strategy. Optimizing an HPC code ecosystem demands careful evaluation of how different techniques overlap and complement one another. Were this a commercial or funded research project, benchmarking the performance of the various technique combinations (and against the system hardware available) would be prudent.

Figure 16 illustrates our parallel processing architecture, which leverages a central Master Node to efficiently orchestrate tasks across the system. With combined optimization and parallelization techniques, a possible workflow functions as follows:

- The Master Node checks if an equilibrated geometry already exists, allowing it to bypass the computationally heavy equilibration phase for previously processed configurations (such as Conf 4, which immediately unlocks its production runs).
- For new configurations (like Confs 1 and 5), the system simultaneously deploys independent replicas.
- These replicas utilize a collaborative swarm logic, aided by the Geweke Convergence test running on 2 workers with 2 threads each, to dynamically find equilibrium as fast as possible.
- Since an equilibrated geometry for conformation type 4 is already supplied, the 2 workers with their 2 OpenMP threads each are immediately available to assist in the production runs for that configuration.
- Meanwhile equilibration completes for conformations 4 & 5, and those workers become available to start assisting in the remaining jobs from the shared 30-job global production queue.

Total core count for this scenario (default when the parallel submission script is invoked) is **13 cores**:

- **1** Master Node (**1 core**)
- 2 Workers each running a Conformation Equilibration
  - Each of these workers takes 2 cores for Swarm Logic
    - Each node calculates energy via 2 OpenMP threads ($2 \times 2 \times 2 =$ **8 cores**)

- 2 Workers immediately start production runs on equilibration-bypassed conformation #4. (*Why 2? The system opts to utilize the same number of workers for a bypassed configuration as it would a swarm-logic equilibration.*)
  - These workers' energy calculations are threaded with OpenMP ($2 \times 2 =$ **4 cores**)
- When the equilibration phase is done for conformations 1 & 5, those workers are reassigned unfinished chunks of the production queue, *reusing the CPU resources*, 4 cores per conformation. (+**0 cores**)
- As workers finish their subtasks, they inform the orchestrator, receiving any pending jobs from the global queue until all 30 production jobs are completed. (+**0 cores**)

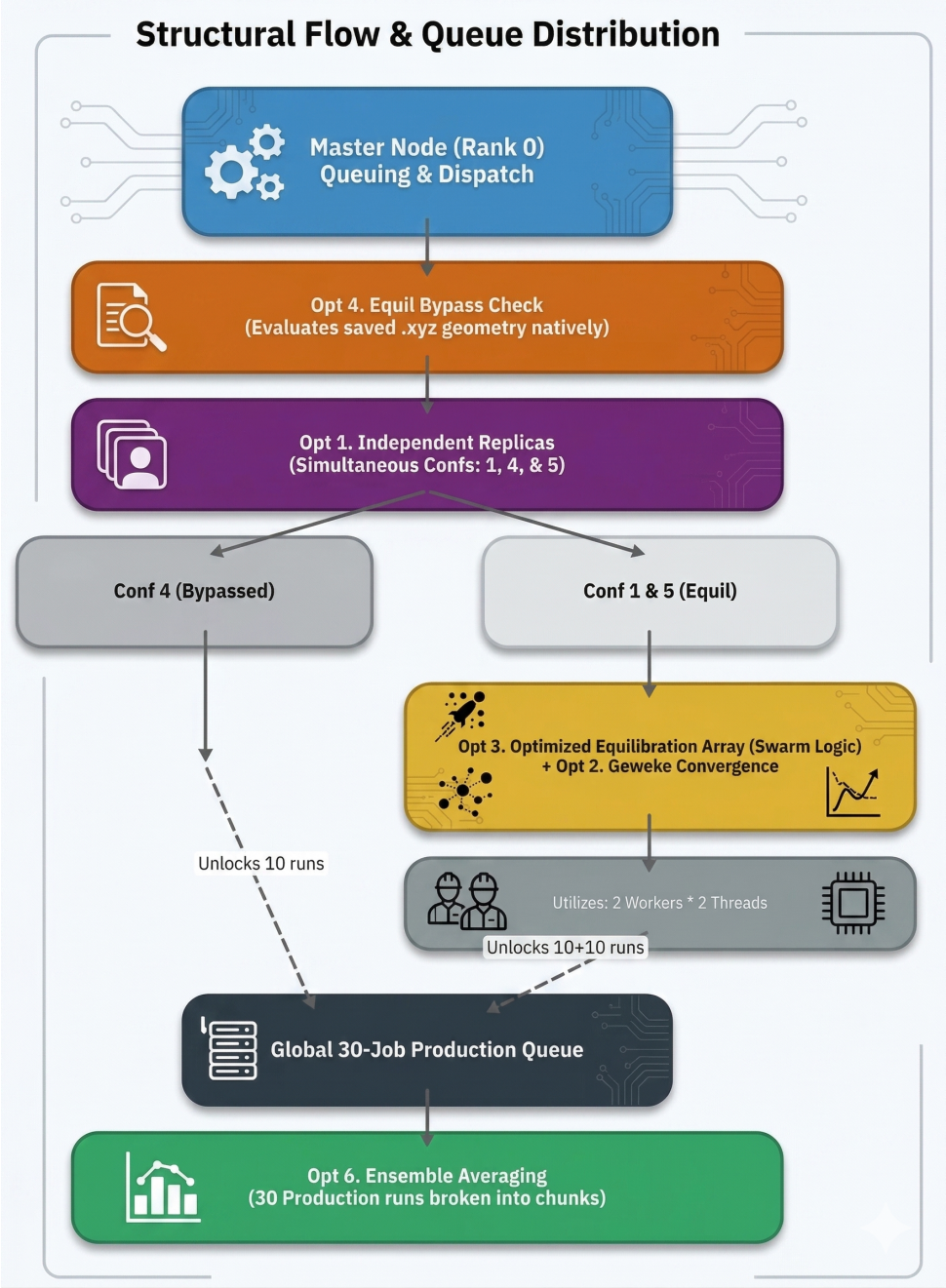

Figure 16: Schematic view of the combined MPI/OpenMP resource utilization strategy.

12.5 Wall Clock Comparison: Parallel vs Serial

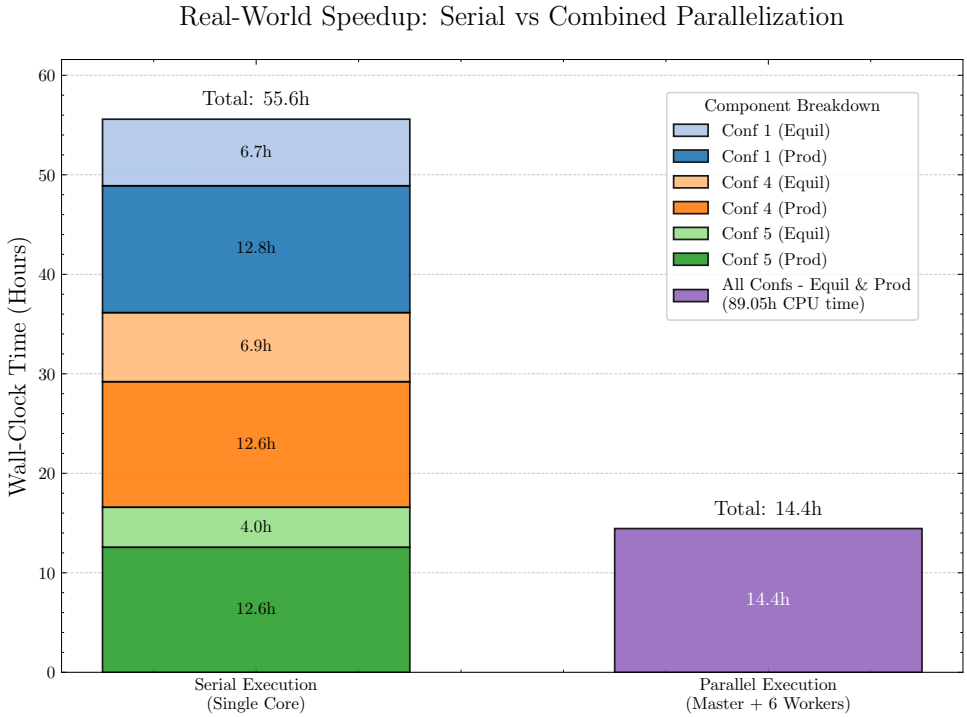

Figure 17: Wall-Clock timing for serial vs combined parallelized simulation

A final test of the parallelization efforts' efficiency was running the simulation serially, combining their runtimes, and comparing it to the wall-clock time required to complete the same parallel execution. No equilibration steps were bypassed to keep the comparison as similar as possible.

As seen in figure 17, the parallel version took 14h 26min to complete while the serial versions took a combined 55h 35m to perform the same tasks. The parallel version required 13 cores to operate, consisting of over 89h of compute time, not counting the master node's orchestration. The multi-processing approach condensed the parallel version's CPU-to-Wall clock time by a factor of **6.16x**, while the wall-to-wall clock speedup (parallel vs serial) was sped up by **3.85x**.

It's interesting to notice that while more resources are needed to run the parallel version, it was also able to complete all 3 configurations' equilibration and production simulations faster than the quickest serial simulation (Conf 5). This beautifully illustrates the contributions of high-performance computing architecture to scientific research, showing that it can assist in converting multi-day research bottlenecks into overnight jobs.

References

- Chodera, J.D. (2016). A simple method for automated equilibration detection in molecular simulations. *J. Chem. Theory Comput.*, 12(4), 1799–1805. doi:10.1021/acs.jctc.5b00784
- Roy, V. (2020). Convergence Diagnostics for Markov Chain Monte Carlo. *Annual Review of Statistics and Its Application*, 7, 387–412. doi:10.1146/annurev-statistics-031219-041300
- Martin, M.G. and Siepmann, J.I. (1998). Transferable Potentials for Phase Equilibria. 1. United-Atom Description of *n*-Alkanes. *J. Phys. Chem. B*, 102(14), 2569–2577. doi:10.1021/jp972543+
- Sæther, S., Falck, M., Zhang, Z., Lervik, A. and He, J. (2021). Thermal Transport in Polyethylene: The Effect of Force Fields and Crystallinity. *Macromolecules*, 54(13), 6563–6574. doi:10.1021/acs.macromol.1c00633